

적층제조 출력 방향을 위한 빌드플레이트 기반 고속 후보 생성과 슬라이서 Warm-Start: Support Flow Tensor Field 접근

설인환

Abstract

적층제조에서 지지구조량은 출력 방향에 크게 좌우되므로, 지지구조를 최소화하는 출력 방향을 찾는 것이 중요하다. 저자들의 선행 GPU 기반 지지구조 단층촬영(GPU-MSST, 본 논문에서 TOMO.INT3로 지칭)은 yaw-pitch 전역 탐색으로 정확한 지지구조량을 주지만, 단일 방향 평가가 빨라졌을 뿐 129,600개 방향을 전수 평가하므로 형상 복잡도와 무관한 큰 고정비용을 갖는다. 본 연구는 이 전수 탐색을 대체하는 것이 아니라, 그 앞단에서 유망 방향을 빠르게 제안하는 후보 생성기 겸 slicer/TOMO warm-start를 제안한다. 구체적으로 구면상 후보 방향마다 저비용 지지 비용을 평가하여 소수의 유망 배향을 추려 내고, 이를 TOMO.INT3나 상용 슬라이서의 국소 검증에만 입력함으로써 전체 탐색 비용을 줄인다.

본 연구는 3D프린팅의 지지구조 추정을 *방향의존* 문제로 보고, 방향별 비용을 방향 n 에 대해 직접 평가하는 *비대칭 흐름 텐서*(Support Flow Tensor Field) $F(n)$ 을 골격으로 구성한다. 이는 법선 분포만 담는 기존의 대칭 텐서(baseline “Support Tensor”)와 달리, 오버행 면에서 그 아래의 실제 받침면으로 향하는 면-면 지지 흐름을 ray casting으로 직접 반영한다. 실제 지지구조량과의 상관 분석 결과, 예측력의 대부분은 빌드플레이트 지지 비용과 면-면 내부 가시성 향에서 비롯되며, nuclear norm 등 고차 텐서 항의 기여는 작았다. 나아가 빌드플레이트 지지 비용이 흐름 텐서에 가상 접지 노드(법선이 빌드 방향인 극한 받침면)를 더한 Rayleigh 기여와 정확히 동치임을 보여($\max |\hat{R} - (R + B)| < 10^{-12}$), 예측력을 주도하는 항 자체가 단일 텐서 수축으로 표현됨을 확인하였다(접지 노드를 통한 접지통합, 통합 Rayleigh cost $\hat{R} = R + B$). 이 통합 항은 음의 신호로 작용하는 면-면 지지량 항을 제외함으로써 기존 합산 목적함수보다 일관되게 높은 예측력을 보인다.

저장된 TOMO.INT3 grid에 대한 상관 분석, ablation, Leave-One-Mesh-Out 교차검증에서 제안 방법은 PCA 및 Support Tensor 고유벡터 기반 방향보다 일관되게 우수하였다. 또한 난이도를 통제한 5-그룹 형상 집합(기본 도형부터 $\sim 10^6$ 면 대형 메쉬까지, 3° 전수 sweep 기준)에서 Python SFTF는 TOMO.INT3 대비 1.4–34 $\times$, C++ SFTF는 10–5,160 $\times$ 의 계산 시간 단축을 보였다. 또한 TOMO.CUDA가 단순 형상에서는 INT3보다 빠르지만 실제 부품과 대형 메쉬에서는 항상 우세하지 않음을 확인하여, 핵심 이득이 전수 sweep backend 변경보다 후보 수 축소에 있음을 보였다. 나아가 임계각 $\theta_c \in \{0, 30, 45, 60, 90\}^\circ$ 전반에서 TOMO 최적 배향과 지지체적의 변화를 측정하여, 지지체적이 θ_c 증가에 따라 단조 감소함과 동시에 전역 최적 배향이 이동함을 확인하였다. 끝으로 coarse 구면 샘플링 후 우수 후보 주변을 소각도로 재샘플링하고, 단일 최적점이 아니라 서로 떨어진 여러 near-optimal 배향군(basin)을 함께 제시한다.

핵심어: 3D프린팅, 지지구조, 비대칭 흐름 텐서, 방향의존, 접지통합

1 서론

1.1 출력 방향과 지지구조 문제

적층제조(additive manufacturing), 특히 가장 널리 쓰이는 FDM(fused deposition modeling) 방식에서는 용융된 필라멘트를 한 층씩 쌓아 올린다 [8]. 이때 임계각(critical angle, θ_c)보다 더 가파른 하향 오버행(overhang)을 출력하려면 그 아래를 받쳐 줄 지지구조(support structure, SS)가 필요하다. 지지구조는 출력이 끝난 뒤 제거되는 희생 구조이지만, 추가 필라멘트와 출력 시간을 소모하고, 제거 자국과 계단 현상(staircasing)으로 표면 품질을 떨어뜨린다. 따라서 지지구조량은 FDM 출력의 비용과 품질을 좌우하는 핵심 인자이다.

같은 형상이라도 빌드플레이트 위에 어떤 자세로 안착시키느냐에 따라 오버행의 분포가 달라지므로, 출력 방향(build orientation)은 다음 요소에 직접 영향을 준다.

- 지지구조량
- 출력 시간
- 재료 사용량
- 표면 품질
- 출력 안정성

이 가운데 지지구조량을 최소화하는 출력 방향을 찾는 것이 본 연구가 다루는 최적 방향 탐색 문제이다.

1.2 관련 연구

출력 방향 최적화와 지지구조 저감은 적층제조 설계에서 오래 다루어진 주제로, 선행 연구의 접근은 크게 (i) 지지구조량이 적은 출력 방향을 찾는 *방향 최적화*, (ii) 주어진 방향에서 지지구조의 형상을 최적화하는 *지지구조 설계*, (iii) 부품의 형상·위상을 바꾸어 지지 요구 자체를 줄이는 *설계 단계 접근*으로 나눌 수 있다.

방향 최적화에서 Das 등은 메쉬를 복셀화하여 지지구조량과 부품 오차를 함께 고려하는 출력 방향을 선택하였고 [4], Wang 등은 layered depth-normal image 표현과 GPU 가속으로 복잡한 다면체의 기하 연산을 빠르게 처리하여 복셀 기반 적층제조 계산의 토대를 마련하였다 [30]. 이러한 명시적·복셀 기반 평가는 단일 방향의 지지구조량을 정확히 주지만, 방향을 전수 탐색할 때에는 형상 복잡도와 무관하게 큰 고정비용을 수반한다 (§13에서 정량적으로 보인다).

지지구조 설계 측면에서는 격자(lattice)나 셀룰러 구조로 지지 부피를 최소화하는 연구가 활발하다. Vaissier 등은 유전 알고리즘으로 격자 지지구조를 가지치기하여 재료 소모를 줄였고 [29], Jiang 등은 압출식 적층제조에서 출력 가능한 임계 오버행 각도를 실험적으로 규명하여 불필요한 지지를 저감하였다 [14]. 설계 단계에서 지지 요구를 줄이려는 시도로, Mirzendehtdel과 Suresh는 지지구조를 제약으로 포함한 위상 최적화를 제안하였으며 [21], Langelaar는 부품 위상, 지지 배치, 출력 방향을 동시에 최적화하는 통합 정식화를 제시하였다 [19]. 한편 Dai 등은 다축 운동을 이용해 지지 자체가 필요 없는 출력을 달성하는 대안 패러다임을 보였다 [3].

특히 오버행의 방향 의존성을 *이방성(anisotropy)*이나 텐서로 직접 표현하려는 흐름이 위상 최적화(topology optimization) 진영에서 발전해 왔으며, 이는 본 연구의 텐서 정식화와 가장 가까운 계열이다. Gaynor 등은 SIMP 위상 최적화에 최대 오버행 제약을 도입하여, 각 층이 자기지지 각도 안에서 아래의 솔리드에 의해 받쳐지도록 강제하였다 [7]. Garcke 등은 phase-field 위상 최적화에서 오버행 경계를 이방성 계면 에너지 (anisotropic interface energy, Wulff 형상)로 벌점화하여, 자기 지지 가능한 경계 법선 방향이 선호되도록 만들었다 [6]. Shin 등은 다축(multi-axis) 적층제조를 대상으로 증착 방향에 따른 재료 강성 이방성 텐서 $\mathbf{D}(\phi) = \mathbf{T}(\phi) \mathbf{D}_0 \mathbf{T}(\phi)^T$ 와 오버행 벌점, 그리고 단계별 빌드 방향을 동시에 최적화하였다 [25]. 산업 측면에서는 Smith와 Hollenbeck이, 직교면과 내부 면이 많아 지지 제거가 어려운 복잡 금속 부품의 빌드 방향을 선택하여 지지구조와 표면 왜곡을 최소화하는 방법을 특허화하였다 [26].

본 연구도 텐서로 오버행의 방향성을 담는다는 점에서 위 이방성 계열과 발상을 공유하지만, 그 위치는 상보적이다. 위 연구들의 텐서는 부품의 위상·밀도를 바꾸는 PDE 제약 설계 최적화 안에서 재료 강성(Shin)이나 계면 에너지(Garcke)의 물리적 이방성을 나타내며, 빌드 방향은 고정(Gaynor, Garcke)되거나 설계 변수와 함께 최적화된다(Shin). 반면 본 연구의 Support Flow Tensor Field는 고정된 부품에 대해 후보 방향마다 평가하는 순수 기하적 지지 흐름 기술자로, 부품을 재설계하지 않고 빌드 방향을 저비용으로 *순위화*하는 데 쓰인다. 즉 본 연구는 지지구조의 형상이나 부품 위상을 바꾸지 않고 표준 FDM 단일 방향 출력을 가정한 채 *출력 방향*에 집중하되, 고비용 전수 평가를 대체하기보다 그 앞단에서 유망 방향을 저비용으로 추려 정확한 검증기나 상용 슬라이서의 국소 검증으로 넘기는 후보 생성기 겸 warm-start를 지향한다. Smith와 Hollenbeck의 특허 [26]는 목표(고정 형상의 빌드 방향 선택)는 본 연구와 가깝지만 절차가 공개되지 않은 산업적 방법인 반면, 본

연구는 투명한 텐서 비용과 검증기 warm-start를 제공한다. 다음 절에서는 본 연구가 직접 확장하는 암시적 추정 계열(그림자 부피 기반 MSST)을 먼저 정리한다.

1.3 최적 방향 탐색: 명시적/암시적 방법과 MSST

최적 출력 방향은 명시적(explicit) 또는 암시적(implicit)으로 구할 수 있다. 명시적 방법은 각 후보 방향마다 실제로 슬라이싱하여 G-code를 생성하고 지지구조량을 직접 계산한다. 정확하지만 비용이 커서, 선행 연구들은 메쉬를 voxel화하고 GPU로 G-code 생성을 가속하였음에도 계산 부담 때문에 대개 단일 방향 평가에 머물렀고, 최적 방향 전수 탐색에는 적용되지 못하였다.

암시적 방법은 복잡한 G-code 생성 없이 지지구조량을 간접적으로 추정한다. 대표적으로 Ezair 등은 지지구조의 부피가 가상 태양광이 만드는 그림자 부피와 유사함을 관찰하였다 [5]. 직관적이지만 임계각이 0° 인 경우, 즉 모든 오버행이 지지를 필요로 하는 경우에만 성립한다. 본 저자들의 선행 연구 [16]는 이 아이디어를 임의의 임계각으로 일반화하여 “modified support structure tomography (MSST)”로 발전시켰다.¹

MSST는 가상 광선을 다수의 ray로 나누고, 각 ray가 표면과 만나는 교점을 “pixel”로 본다. 같은 (x, y) 위치의 pixel을 Z좌표로 정렬한 “pixel slot” 안에서 위를 향하는 상향면(α)과 아래를 향하는 하향면(β)을 분류하고, $V_{ss} = V_{tc} - V_o - V_{nv}$ 형태로 top-cover 부피(V_{tc})에서 객체 부피(V_o)와 비오버행 부피(V_{nv})를 빼서 지지구조 부피(V_{ss})를 추정한다. 이 방식은 Stanford Bunny 등 여러 표준 메쉬에서 상용 슬라이서와 거의 같은 지지구조량을 재현하였다.

1.4 GPU-MSST와 남은 한계

MSST의 유일한 약점은 속도였다. 65k 삼각형 Bunny의 단일 방향 계산에 약 1초가 걸리고, yaw-pitch를 1° 간격으로 훑으면 $360 \times 360 = 129,600$ 회의 단일 방향 계산이 필요하여, CPU에서 한번의 최적 방향 탐색에 1-2분이 걸린다. 본 저자들은 최근 MSST를 GPU 친화적 자료구조(slot 기반 pixel 자료구조, 삼각형 분할 voxel화)로 재구성한 GPU-MSST를 제안하여 [18], 동일한 전수 탐색을 약 10초로 단축하고 고급 CPU 대비 평균 11.2배, 저가 CPU 대비 최대 62배의 가속을 보고하였다. 본 논문에서는 이 GPU 기반 MSST 구현을 정확한 지지구조량 검증기로 사용하며 TOMO_INT3로 지칭한다.

그러나 GPU-MSST 역시 본질적으로 전체 yaw-pitch grid를 전수 탐색한다. 즉 가속은 단일 방향 평가를 빠르게 만든 것이지, 평가해야 할 방향의 개수(129,600)를 줄인 것이 아니다. 그 결과 탐색 비용은 형상의 복잡도와 거의 무관하게 큰 고정비용을 가지며 (본문 §13에서 면이 12개뿐인 cube조차 3° TOMO_INT3 sweep에 약 4초가 걸린다), 간단한 형상이나 빠른 사전 검토에는 과도하다. 실제로 선행 연구 스스로도 “ α 면과 β 면 사이의 제한된 부분만이 최종 지지구조에 기여한다”는 관찰에서 출발하여, 평가 대상을 줄이는 텐서적 접근(잠정 명칭 shadow tensor)을 향후 과제로 제시한 바 있다. 본 연구는 바로 그 방향의 후속 작업에 해당한다.

1.5 본 연구: Support Flow Tensor Field

본 연구의 목표는 고비용 전수 탐색을 대체하는 것이 아니라, 그 앞단에서 유망한 출력 방향을 빠르게 제안하는 후보 생성기(candidate generator) 겸 slicer/TOMO warm-start를 만드는 것이다. 구면상의 후보 방향마다 저비용 지지 비용을 평가하여 소수의 유망 배향만 추려 내고, 이를 TOMO_INT3나 상용 슬라이서의 국소 검증에 입력함으로써 전체 탐색 비용을 줄인다.

이를 위해서는 형상을 요약하는 텐서가 필요하다. 가장 단순한 후보는 면 법선의 분포를 면적 가중으로 모은 대칭 텐서 $M = \sum_i A_i m_i m_i^T$ 이며, 본 논문에서는 이를 비교 기준(baseline)으로서 “Support Tensor”라 부른다. 그러나 §2에서 보이듯 이 대칭 텐서는 법선 분포만 담을 뿐, 지지구조가 실제로 어디에서 어디로 흐르는지를 설명하지 못한다. 특히 오목(concave) 영역, 내부 차폐(occlusion), 빌드플레이트까지의 높이, 출력 방향에 따른 가시성 변화가 반영되지 않는다.

따라서 본 연구는 방향 $n \in S^2$ 에 대해 *비대칭* Support Flow Tensor Field $F(n)$ 을 직접 평가한다 (§3). $F(n)$ 은 오버행 면에서 그 아래의 실제 receiver 면으로 향하는 면-면 지지 “흐름”을 ray

¹본 저자들의 MSST [16] 및 GPU 가속(GPU-MSST) [18] 선행 연구. 공개 소스코드: https://github.com/cfms-lab/Tomo_GPU2024.

casting으로 찾아 누적한 것으로, 단순 고유벡터 문제가 아니라 방향별 비용 평가 문제로 다룬다. 또한 본 연구의 핵심 통합 관점으로, 빌드플레이트 지지 비용을 별개의 스칼라 penalty가 아니라 흐름 텐서에 “법선이 빌드 방향인 가상 접지 노드(ground node)”를 추가했을 때의 Rayleigh 기여로 정식화한다. 이로써 면-면 자기지지와 빌드플레이트 지지가 하나의 흐름 텐서로 통합되며 (통합 Rayleigh cost $\tilde{R} = R + B$, §5.1), 정량 분석 (§12.2)에서 이 통합 항이 예측력을 주도함을 확인한다. 요약하면 본 연구의 기여는 다음과 같다.

- 출력 방향별로 비대칭 Support Flow Tensor Field $F(n)$ 을 정의하고, 면-면 내부 가시성을 ray casting으로 직접 평가한다.
- 빌드플레이트 지지 비용이 흐름 텐서에 가상 접지 노드를 더한 Rayleigh 기여와 정확히 동치임을 보여(식 (18)), 면-면 자기지지와 빌드플레이트 지지를 단일 텐서로 통합한다.
- 저장된 TOMO_INT3 grid에 대한 상관 분석, ablation, 교차검증으로 예측력의 출처를 규명하고, PCA 및 Support Tensor 고유벡터 baseline 대비 우위를 정량 검증한다.
- 난이도를 통제된 5-그룹 형상 집합($\sim 10^6$ 면 대형 메쉬 포함)에서 Python SFTF와 C++ SFTF를 TOMO_INT3/TOMO_CUDA 3° 전수 sweep과 비교하여, TOMO_INT3 대비 각각 1.4–34× 및 10–5,160×의 계산 시간 단축과 단일 v_{ss} 가 제공하지 못하는 방향별 비용 구조 해석을 제시하고, 임계각 $\theta_c \in \{0, 30, 45, 60, 90\}^\circ$ 전반에서 TOMO 최적 배향과 지지체적의 변화를 분석한다.

2 Baseline인 (대칭) Support Tensor와 그 한계

용어 정리 (중요). 본 논문에는 이름이 비슷한 두 텐서가 등장하므로 먼저 구분해 둔다. *Support Tensor*는 면 법선의 분포를 면적 가중으로 모은 **대칭** 텐서로, 본 연구가 새로 제안하는 것이 **아니라** 비교 기준(baseline)으로 사용하는 고전적 구성이다. 이는 섬유/입자 역학의 orientation tensor (또는 fabric tensor)나 영상 처리의 structure tensor와 같은 계열, 즉 “법선 벡터들의 2차 모멘트”에 해당한다.² 반면 본 연구가 새로 제안하는 것은 다음 절 (§3)의 방향 의존 **비대칭** 텐서인 *Support Flow Tensor Field* $F(n)$ 이다. 요컨대 “Support Tensor”는 기존 baseline, “Support Flow Tensor Field”가 본 연구의 기여이다. 한편 위상 최적화에서 오버행을 다루는 이방성 계면 에너지나 강성 텐서 [6, 25]는 이 둘과 또 다른 범주로, 고정 형상의 방향 비용을 재는 기술자가 아니라 부품을 재설계하는 설계 최적화의 물리적 이방성 모델이다 (§1.2).

정의와 성질. Baseline인 (대칭) Support Tensor는

$$M = \sum_i A_i m_i m_i^T \quad (1)$$

로 정의된다. 여기서 A_i 는 삼각형 면 i 의 면적, m_i 는 외향 법선이다. M 은 대칭 양의 준정부호 (positive semidefinite) 행렬³ 이므로 고유벡터(eigenvector)⁴ 해석이 가능하다. 즉 M 의 고유벡터는 형상의 법선이 평균적으로 어느 방향을 향하는지(주방향)를 알려 준다.

한계. 그러나 M 은 법선의 분포만 담을 뿐, 지지구조가 실제로 생성되는 과정을 담지 못한다. 구체적으로 M 은 다음 정보를 포함하지 못한다.

- 특정 방향에서 어떤 면이 오버행인지
- 오버행 면 아래에 실제로 지지 가능한 다른 면이 있는지

²대칭 텐서 $\sum_i A_i m_i m_i^T$ 는 법선 m_i 들의 (면적 가중) 외적 합으로, structure tensor / orientation tensor 계열의 고전적 형상 기술자이다. 참고: [17].

³모든 벡터 x 에 대해 $x^T M x \geq 0$ 인 대칭 행렬. 고유값이 모두 ≥ 0 이라는 뜻이다. 참고: [13]

⁴ $Mx = \lambda x$ 를 만족하는 방향 x 로, 행렬이 가장 강하게 “늘이는” 축들을 나타낸다. 참고: [9]

- 지지 대상까지의 높이 또는 거리
- 빌드플레이트가 직접 받쳐야 하는 영역
- 비대칭적인 면-면 지지 흐름

따라서 Support Tensor의 고유벡터는 형상의 통계적 방향성은 설명할 수 있으나, 슬라이서가 계산하는 지지구조량의 최적 방향과 항상 일치하지 않는다. 이 한계가, 다음 절에서 방향별 비대칭 흐름 텐서를 도입하는 출발점이다.

3 방향별 Support Flow Tensor Field

출력 방향을 단위벡터

$$n \in S^2, \quad \|n\| = 1 \quad (2)$$

이라 한다. 면 i 의 중심을 c_i , 외향 법선을 m_i , 면적을 A_i 라 하면, 방향 n 에서의 오버행 계수는

$$O_i(n) = \max(0, -m_i \cdot n) \quad (3)$$

으로 정의한다.

오버행 면 i 에서 방향 $-n$ 으로 ray를 발사하여 가장 먼저 만나는 유효 면을 j 라 한다. 이때 높이 차이는

$$h_{ij}(n) = (c_i - c_j) \cdot n \quad (4)$$

이며, $h_{ij} > 0$ 인 경우만 지지 흐름으로 인정한다.

면-면 영향도는

$$w_{ij}(n) = \frac{O_i(n)}{1 + \alpha h_{ij}(n)} \quad (5)$$

으로 둔다. 현재 구현에서는

$$\alpha = \frac{1}{D} \quad (6)$$

를 사용하며, D 는 입력 메쉬의 bounding box diagonal이다.

따라서 방향 n 에 대한 Support Flow Tensor는

$$F(n) = \sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T \quad (7)$$

로 계산한다. 여기서 $\mathcal{R}(n)$ 은 ray casting⁵ 으로 얻어진 유효한 면-면 지지 관계의 집합이다. $F(n)$ 은 $m_i m_j^T$ 의 합이므로 일반적으로 대칭 행렬이 아니다.

⁵광선(ray)을 쏘아 가장 먼저 만나는 표면을 찾는 기법. 여기서는 오버행 면에서 빌드 방향의 반대($-n$)로 광선을 쏘아 아래의 받침면을 찾는 데 쓴다. 참고: [23]

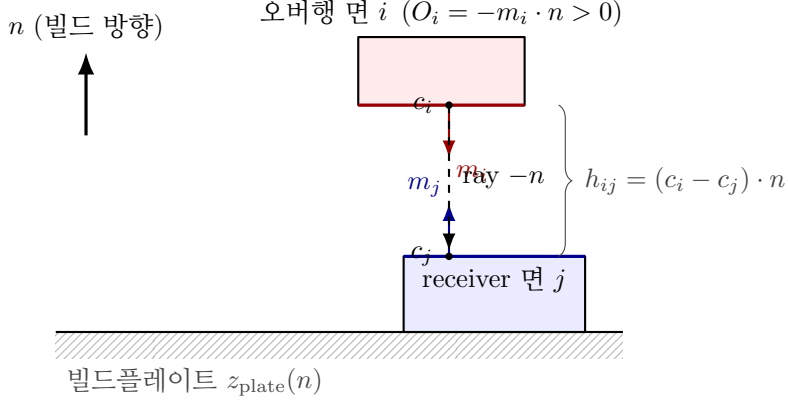


Figure 1: 지지 흐름의 기하. 오버행 면 i (아래를 향한 법선 m_i , $O_i = \max(0, -m_i \cdot n) > 0$)에서 빌드 방향의 반대 $-n$ 로 ray를 쏘아 아래의 유효 receiver 면 j 를 찾는다. 높이차 $h_{ij} = (c_i - c_j) \cdot n > 0$ 일 때만 지지 흐름으로 인정하며, 이 면-면 쌍이 텐서 $F(n) = \sum w_{ij} A_i A_j m_i m_j^T$ 의 한 항을 이룬다.

4 Ray Casting과 유효 지지 관계

각 후보 방향 n 에 대해 모든 오버행 면을 전부 ray casting하면 계산량이 커진다. 따라서 현재 구현에서는 오버행 기여도

$$A_i O_i(n) \quad (8)$$

에 비례하도록 source face를 결정론적 stratified sampling⁶ 으로 선택한다. ray 개수는 면 개수 N_f 에 따라

$$K_{ray} = \text{clip}(0.02N_f, 500, 3000) \quad (9)$$

으로 정한다.

Ray origin은 자기 교차를 피하기 위하여

$$r_i(0) = c_i - \varepsilon n \quad (10)$$

으로 약간 이동시키며, ray 방향은 $-n$ 이다. 현재 구현에서는

$$\varepsilon = 10^{-6} D \quad (11)$$

를 사용한다.

유효한 receiver face j 는 다음 조건을 만족해야 한다.

1. $j \neq i$ 이다.
2. ray 거리가 충분히 양수이다.
3. $h_{ij}(n) > 10^{-5} D$ 이다.
4. receiver normal이 빌드 방향을 충분히 향한다.

마지막 조건은

$$m_j \cdot n > \eta \quad (12)$$

로 구현되며, 현재 $\eta = 0.05$ 를 사용한다. 이는 ray가 닿았더라도 실제로 받침면으로 보기 어려운 수직 또는 뒤집힌 면을 제거하기 위한 조건이다.

⁶층화 추출. 전체를 여러 구간(층)으로 나눈 뒤 각 층에서 표본을 뽑아, 단순 무작위 추출보다 고르게 대표 표본을 얻는 방법(층화 추출). 참고: [2]

5 Build Plate 항

Ray casting으로 아래쪽 지지 면을 찾지 못한 오버행 면은 결국 빌드플레이트 또는 별도 지지구조에 의해 받쳐져야 한다. 현재 구현에서는 이 항을 스칼라로 계산하지만, §5.1에서 보이듯 이는 흐름 텐서에 빌드플레이트를 가상 receiver로 추가했을 때의 Rayleigh 기여와 정확히 같다.

방향 n 에 대한 빌드플레이트 높이 기준은

$$z_{plate}(n) = \min_{v \in V} v \cdot n \quad (13)$$

으로 정의한다. 면 i 의 빌드플레이트까지 높이는

$$h_{iP}(n) = c_i \cdot n - z_{plate}(n) \quad (14)$$

이다.

유효한 면-면 hit가 없는 source face 집합을 $\mathcal{B}(n)$ 이라 하면, build-plate support penalty는

$$B(n) = \sum_{i \in \mathcal{B}(n)} O_i(n) A_i (1 + \alpha h_{iP}(n)) \quad (15)$$

으로 계산한다. 이 항은 높이가 높은 오버행이 빌드플레이트까지 직접 지지를 요구할수록 비용을 증가시킨다.

5.1 빌드플레이트의 가상 접지 노드 해석

$B(n)$ 은 텐서 밖에서 더해진 스칼라 penalty처럼 보이지만, 실제로는 면-면 흐름 텐서 $F(n)$ 에 빌드플레이트를 가상 receiver(접지 노드)로 추가했을 때의 Rayleigh 기여와 동치이다. 면-면 흐름 항 $m_i m_j^T$ 에서 receiver 법선 m_j 는 $m_j \cdot n > \eta > 0$ 으로 위를 향하며, 빌드플레이트의 지지면 법선은 정확히 빌드 방향 n 이다 ($m \cdot n = 1$, 즉 가장 위를 향하는 극한 receiver). 따라서 플레이트 지지를 받는 면 $i \in \mathcal{B}(n)$ 을 “법선이 n 인 가상 노드와의 흐름 쌍”으로 보아 다음과 같이 증강 텐서를 정의한다.

$$\tilde{F}(n) = \underbrace{\sum_{(i,j) \in \mathcal{R}(n)} w_{ij}(n) A_i A_j m_i m_j^T}_{\text{면-면 흐름}} + \underbrace{\sum_{i \in \mathcal{B}(n)} A_i (1 + \alpha h_{iP}(n)) m_i n^T}_{\text{빌드플레이트 흐름(가상 노드)}}. \quad (16)$$

플레이트 항의 Rayleigh 기여는 $m_i \cdot n = -O_i(n)$ 이므로

$$n^T \left[\frac{1}{2} (m_i n^T + n m_i^T) \right] n = (m_i \cdot n) (n \cdot n) = -O_i(n) \quad (17)$$

이고, 계수 $A_i(1 + \alpha h_{iP})$ 를 곱해 합산하면 플레이트 항이 $-n^T \tilde{F}_s n$ 에 기여하는 양은 정확히 $B(n)$ 이 된다. 따라서 증강 텐서의 Rayleigh support cost는

$$\tilde{R}(n) = \max(0, -n^T \tilde{F}_s(n) n) = R(n) + B(n) \quad (18)$$

으로 정확히 성립한다. 이 동치는 저장된 5개 메쉬의 후보 풀 전체(각 약 1,292개 후보)에서 $\max |\tilde{R} - (R + B)| < 10^{-12}$ 로 수치 확인되었다(§12.2).

이 재구성의 의의는 두 가지다. 첫째, 예측력의 대부분을 차지하는 빌드플레이트 비용(§12.1)이 ad-hoc 스칼라가 아니라 흐름 텐서의 한 항임이 드러나, “텐서” 정식화가 정당해진다. 둘째, 빌드플레이트는 별개의 메커니즘이 아니라 “면-면 자기지지의 극한(완전히 수평인 receiver)”으로 통합된다—— 모든 오버행은 receiver로 흐르며, 그 receiver는 다른 면이거나 빌드플레이트이다. 높이 규약의 비대칭은 의도적이다. 면-면 가중치는 감쇠 $1/(1 + \alpha h)$ (면 자기지지는 약하게), 플레이트 항은 증폭 $(1 + \alpha h)$ (키 큰 지지기둥은 비싸게)를 쓴다. 이 부호 차이가 임의적이지 않음은 §12.2에서 다른 규약과 정량 비교하여 검증한다.

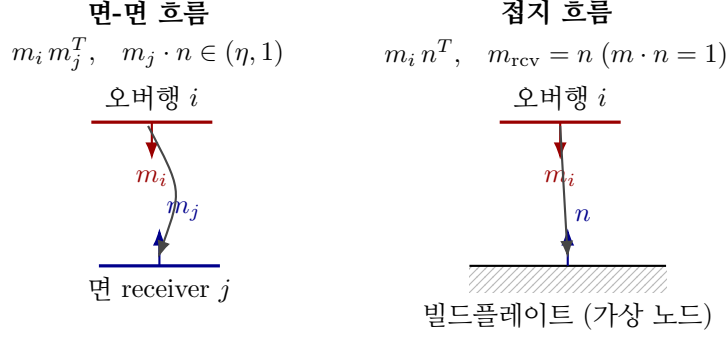


Figure 2: 통합 텐서의 두 흐름 항. (왼쪽) 오버행 i 가 실제 면 j 에 지지되면 receiver 법선은 $m_j \cdot n \in (\eta, 1)$ 이고 텐서 항은 $m_i m_j^T$ 이다. (오른쪽) 받쳐 줄 면이 없으면 빌드플레이트라는 “법선이 정확히 n 인 극한 receiver”($m \cdot n = 1$)에 지지되어 항은 $m_i n^T$ 가 된다. 빌드플레이트 비용 $B(n)$ 은 이 접지 항의 Rayleigh 기여와 정확히 같다(식 (18)).

6 방향별 목적함수

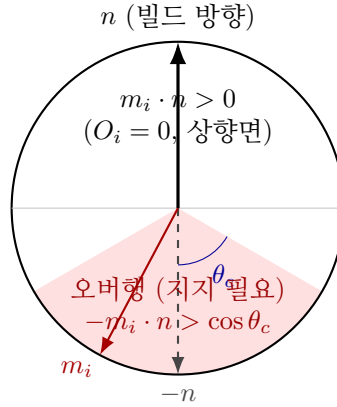


Figure 3: 오버행 계수 $O_i(n) = \max(0, -m_i \cdot n)$ 의 기하적 의미. 가능한 면 법선 방향을 빌드 방향 n 기준의 단위원에 나타냈다. 위 반원($m_i \cdot n > 0$)은 상향면으로 $O_i = 0$ 이고, 아래쪽 음영은 $-n$ 둘레 반각 θ_c 의 콘으로 $-m_i \cdot n > \cos \theta_c$ 인 면이 실제 지지를 요구하는 오버행이다. O_i 는 법선이 $-n$ 에 정렬될수록(아래로 갈수록) 커진다.

비대칭 텐서 $F(n)$ 을 방향 비용으로 사용하기 위해 대칭 성분을

$$F_s(n) = \frac{F(n) + F(n)^T}{2} \quad (19)$$

로 정의한다.

초기 접근에서는 $n^T F_s(n) n$ 을 직접 사용하였으나, 실험 결과 강한 지지 흐름에서 값의 부호가 음수가 되어 정렬 방향이 뒤집힐 수 있었다. 따라서 현재 구현에서는 Rayleigh 항⁷을 비음수 support cost로 변환한다.

$$R(n) = \max(0, -n^T F_s(n) n) \quad (20)$$

⁷Rayleigh 몫(Rayleigh quotient): 대칭 행렬 A 와 벡터 x 에 대해 $\frac{x^T A x}{x^T x}$ 로 정의되는 스칼라이다. 본문은 $\|n\| = 1$ 이므로 $n^T F_s(n) n$ 자체가 방향 n 에서의 Rayleigh 몫이며, 그 부호를 뒤집어 “지지 비용”으로 해석한다. (고유값과 밀접한 양으로, [13])

면-면 지지량 항은

$$P(n) = \sum_{(i,j) \in \mathcal{R}(n)} \left(\frac{O_i(n)}{1 + \alpha h_{ij}(n)} A_i A_j \right) (1 + \alpha h_{ij}(n)) \quad (21)$$

으로 둔다. 여기서 면-면 영향도 w_{ij} 에 포함된 높이 감쇠 인자 $1/(1 + \alpha h_{ij})$ 와 바깥의 $(1 + \alpha h_{ij})$ 가 서로 상쇄되므로, $P(n)$ 은 결국

$$P(n) = \sum_{(i,j) \in \mathcal{R}(n)} O_i(n) A_i A_j \quad (22)$$

로 단순화된다. 즉 $P(n)$ 은 유효 hit에 대한 면적 가중 오버행량 자체를 나타내며, 높이 의존성은 의도적으로 제거하였다. 이는 텐서 $F(n)$ 에는 높이 감쇠를 유지하여 멀리 떨어진 면-면 흐름의 기여를 약화시키되, 면-면 지지량 항 $P(n)$ 은 순수한 자기지지 가능 면적량으로 해석하기 위한 선택이다.

§5.1의 증강 텐서를 사용하면 Rayleigh 항 R 과 빌드플레이트 항 B 가 하나의 통합 Rayleigh cost $\tilde{R} = R + B$ (식 (18))로 합쳐진다. 따라서 최종 목적함수는 통합 텐서 항과 면-면 지지량의 합

$$J(n) = \lambda_R R(n) + \lambda_P P(n) + \lambda_B B(n) = \underbrace{\tilde{R}(n)}_{R+B} + P(n) \quad (23)$$

으로 쓸 수 있으며($\lambda_R = \lambda_P = \lambda_B = 1$), 통합 텐서 관점에서는 “증강 흐름 텐서의 Rayleigh 지지비용 $\tilde{R}$ 에 면-면 지지 면적량 P 를 더한 것”으로 해석된다. §12.2에서 보이듯 $\tilde{R}$ 자체가 J 보다 더 나은 예측자이며, 이는 합산된 P 가 음의 신호로 작용하기 때문이다. 초기 구현에서는 $F(n)$ 의 특이값⁸ $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$ 로부터 nuclear score⁹ $S(n) = \sum_k \sigma_k(n)$ 과 $\sigma_1(n)$ 항을 가중치 0.05로 추가하였으나, §12.1의 ablation에서 이 두 항은 후보 선택을 전혀 바꾸지 않음이 확인되어 목적함수에서 제외하였다. 특이값은 후처리 rank tuning¹⁰의 보조 feature로만 유지한다(§12.3). 최적 출력 방향은

$$n^* = \arg \min_{n \in S^2} J(n) \quad (24)$$

으로 정의한다.

7 구면 샘플링과 국소 재탐색

고유벡터 3개만 사용하는 방식은 Support Flow Tensor Field의 비대칭성과 방향 의존성을 충분히 반영하지 못한다. 따라서 현재 구현에서는 성긴 전역 표본을 평가한 뒤 유망 후보 주변만 조밀하게 재표본하는 coarse-to-fine(국소 재탐색) 전략[참고문헌 필요]을 따르는 다음 두 단계 탐색을 사용한다.

1. Fibonacci sphere [24, 10]¹¹ 방식으로 512개의 coarse 방향을 생성한다.
2. 각 방향에 대해 $J(n)$ 을 계산한다.
3. 상위 12개 후보를 선택하되, 서로 너무 가까운 방향은 제거한다.

⁸특이값(singular value)과 특이값 분해(SVD): 임의의 행렬을 회전-스케일-회전으로 분해할 때 나오는 비음수 스케일 인자들로, 행렬이 각 축으로 얼마나 “늘이는지”를 나타낸다. 대칭이 아닌 $F(n)$ 에는 고유값 대신 특이값을 쓴다. 참고: [9]

⁹핵 노름(nuclear norm): 특이값의 합 $\sum_k \sigma_k$. Schatten 노름의 한 종류로, 행렬의 전체 “크기”를 재는 척도이다 (행렬 노름의 한 종류). 참고: [13]

¹⁰본 논문에서 “rank tuning”은 표준 용어가 아니라 본 연구의 후처리 절차를 가리킨다. 각 feature를 후보 집합 내부 순위로 정규화(0~1)한 뒤(§12.3의 식 참조), 이 순위 정규화 feature들에 선형 가중치를 학습해 후보를 재정렬하는 것을 뜻한다. 순위(rank)와 순위 상관 개념은 [27] 참조.

¹¹황금각($\approx 137.5^\circ$)을 이용해 구면에 점을 거의 균등하게 흩뿌리는 방법으로, 방향 표본을 고르게 얻는 데 흔히 쓰인다 (코드의 137.508° 회전이 이것이다). 구면 위 점의 준균등 분포에 관한 고전적 결과는 Saff와 Kuijlaars [24]를, Fibonacci 격자의 구면 적용과 균등성 분석은 González [10]를 참고하라.

4. 각 상위 후보 주변을 $2^\circ, 4^\circ, 6^\circ, 8^\circ$ cone으로 재샘플링한다.
5. 각 cone 각도마다 방위각 16개를 평가한다.
6. coarse 후보와 refined 후보를 합쳐 다시 정렬한다.
7. 최종적으로 서로 3° 이상 떨어진 상위 3개 방향을 반환한다.

이 방식은 전체 구면을 조밀하게 모두 평가하는 것보다 빠르면서도, 최적 방향 주변의 작은 각도 변화에 대한 민감도를 반영할 수 있다.

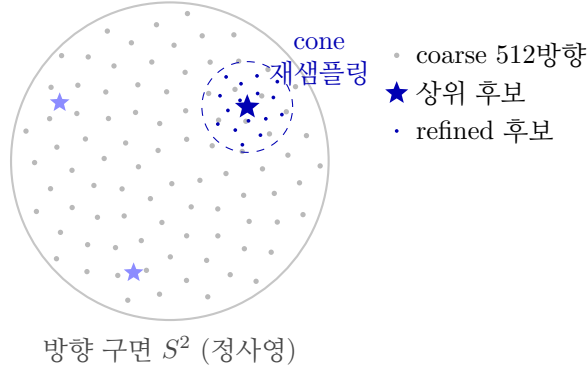


Figure 4: 2단계 방향 탐색. Fibonacci sphere로 구면을 성기게 덮는 coarse 후보(회색, 512개)를 평가한 뒤, 상위 후보(별) 주변을 2° – 8° cone으로 조밀하게 재샘플링(파란 점)하여 최적 방향 부근의 민감도를 반영한다. 전 구면을 조밀 평가하지 않고도 국소 정밀도를 얻는다.

8 계산 가속과 최종 후보 재점수화

방향별 Support Flow Tensor Field는 후보 방향마다 다수의 ray를 발사하므로, 직접 구현할 경우 계산 병목은 대부분 ray-triangle intersection [22]에서 발생한다. 초기 구현에서는 각 ray에 대해 모든 hit를 구한 뒤, 거리순으로 정렬하여 첫 번째 유효 receiver face를 찾았다. 그러나 대부분의 ray에서는 가장 가까운 hit가 곧 유효 hit이므로, 모든 hit를 매번 계산하는 것은 불필요한 비용을 만든다.

현재 구현에서는 먼저

$$\text{multiple_hits} = \text{False} \quad (25)$$

인 first-hit query를 수행한다. 이 first-hit가 자기 면, 너무 짧은 거리, 음의 높이 차, 또는 receiver normal 조건을 위반할 때에만 해당 ray에 대해 기존의 multiple-hit query를 fallback으로 수행한다. 즉 전체 ray 집합 $\mathcal{Q}$ 에 대해

$$\mathcal{Q}_{\text{fallback}} = \{q \in \mathcal{Q} \mid \text{firstHit}(q) \text{ is invalid}\} \quad (26)$$

만 추가 탐색한다. 이 전략은 기존의 정확한 유효 hit 선택 규칙을 유지하면서도, 대부분의 ray에서 intersection 후보 수와 정렬 비용을 줄인다.

또한 후보 방향마다 반복 계산되던 면 법선, 면적, 중심점, vertex 배열, bounding box diagonal D , $\alpha = 1/D$, ray offset ε , height tolerance를 메쉬 단위로 한 번만 계산하여 캐싱하였다. 이를 위해 현재 구현에서는

$$\mathcal{D}_{\text{mesh}} = \{m_i, A_i, c_i, V, D, \alpha, \varepsilon, \varepsilon_h\} \quad (27)$$

를 Support Flow 평가 전체에서 공유한다.

Bunny 1k 메쉬에 대한 프로파일링에서, 기존 방향 평가 시간은 약 15.7초였으나, first-hit fallback과 mesh surface data caching을 적용한 뒤 약 1.5초로 감소하였다. 동일 메쉬에서 선택된 상위 방향, score, hit count는 기존 multiple-hit 방식과 일치하였다. 따라서 이 최적화는 알고리즘의 의미를 바꾸지 않고 계산량만 줄이는 implementation-level acceleration으로 볼 수 있다.

8.1 TOMO 결과 기반 최종 후보 재점수화

원래 목적함수 $J(n)$ 은 후보 생성과 국소 재탐색에는 효과적이었으나, 최종 상위 3개 후보를 고를 때 TOMO_INT3의 실제 support volume v_{ss} 와 항상 잘 정렬되지는 않았다. 특히 $B(n)$ 항이 작은 방향이 항상 실제 지지구조량이 작은 방향은 아니며, 면-면 흐름량과 hit 수가 너무 작은 후보는 실제로는 충분한 self-support 관계를 만들지 못할 수 있다.

따라서 현재 구현에서는 coarse 및 refined 후보 집합을 만든 뒤, 최종 정렬 단계에서 후보 집합 내부 rank-normalized feature를 사용하여 재점수화한다. feature 값을 직접 쓰지 않고 집합 내부 순위로 변환하는 rank(순위) 정규화는 서로 척도가 다른 feature를 robust하게 결합하기 위한 알려진 기법이다[참고문헌 필요]. 각 후보 n_k 에 대해 다음 feature vector를 둔다.

$$x_k = [J_k, R_k, P_k, B_k, H_k, S_k, \sigma_{1,k}]^T, \quad (28)$$

여기서 H_k 는 유효 hit count이다. 각 feature $x^{(l)}$ 에 대해 후보 집합 내부 순위 정규화를

$$\rho_k^{(l)} = \frac{\text{rank}(x_k^{(l)})}{N-1} \quad (0 \leq \rho_k^{(l)} \leq 1) \quad (29)$$

로 정의한다. 값이 작은 후보의 rank가 0에 가깝고, 큰 후보의 rank가 1에 가깝다.

최종 tuning score는

$$T(n_k) = w_J \rho_k^{(J)} + w_R \rho_k^{(R)} + w_P \rho_k^{(P)} + w_B \rho_k^{(B)} + w_H \rho_k^{(H)} + w_S \rho_k^{(S)} + w_1 \rho_k^{(\sigma_1)} \quad (30)$$

으로 둔다. 현재 실험적 구현에서는 저장된 TOMO_INT3 grid와의 후보별 비교를 바탕으로

$$\begin{aligned} w_J &= 0, & w_R &= 0.731, & w_P &= -2.3897, \\ w_B &= 2.1447, & w_H &= 2.3109, & w_S &= -2.2922, & w_1 &= 1.6363 \end{aligned} \quad (31)$$

를 사용하였다. 이 값은 물리 법칙에서 직접 유도된 상수가 아니라, 현재 구현의 feature scale과 TOMO_INT3 validation set에 대한 empirical calibration이다. 따라서 본 논문에서는 이를 최종 이론식이라기보다 TOMO 또는 slicer feedback으로 Support Flow Tensor Field를 보정하는 한 가지 예로 해석한다. 후보 생성 자체는 기존 $J(n)$ 기반으로 수행하고, rank tuning은 최종 후보 선택 단계에만 적용한다. 이는 지나치게 공격적인 재탐색 방향 변경을 피하고, 기존 Support Flow 목적함수의 구조적 의미를 유지하기 위한 선택이다.

9 현재 구현 의사코드

Algorithm 1 Augmented (ground-node) Support Flow Tensor Evaluation with Final Rank Tuning

```

1: for each sampled build direction  $n$  do
2:   Normalize  $n$ 
3:   Compute  $O_i(n) = \max(0, -m_i \cdot n)$  for all faces
4:   Select  $K_{ray}$  source faces by stratified sampling with weight  $A_i O_i(n)$ 
5:   Initialize  $\tilde{F} \leftarrow 0$  ▷ augmented flow tensor (면-면 + 가상 접지 노드),  $P \leftarrow 0$ 
6:   for each sampled source face  $i$  do
7:     Cast ray from  $c_i - \varepsilon n$  along  $-n$ 
8:     Find the first valid receiver face  $j$ 
9:     if valid  $j$  exists then
10:       $h_{ij} \leftarrow (c_i - c_j) \cdot n; \quad w_{ij} \leftarrow O_i(n)/(1 + \alpha h_{ij})$ 
11:       $\tilde{F} \leftarrow \tilde{F} + w_{ij} A_i A_j m_i m_j^T$  ▷ 면-면 흐름
12:       $P \leftarrow P + w_{ij} A_i A_j (1 + \alpha h_{ij})$ 
13:     else
14:       $h_{iP} \leftarrow c_i \cdot n - z_{plate}(n)$ 
15:       $\tilde{F} \leftarrow \tilde{F} + A_i (1 + \alpha h_{iP}) m_i n^T$  ▷ 빌드플레이트 흐름(가상 접지 노드)
16:     end if
17:   end for
18:    $\tilde{F}_s \leftarrow (\tilde{F} + \tilde{F}^T)/2$ 
19:    $\tilde{R} \leftarrow \max(0, -n^T \tilde{F}_s n)$  ▷  $= R + B$ , 식 (18)
20:    $J(n) \leftarrow \tilde{R} + P$ 
21:   (특이값  $\sigma_1, \sigma_2, \sigma_3$ 는 면-면 부분텐서  $F$ 에서 rank tuning 보조 feature로만 계산)
22: end for
23: Generate refined directions around the best coarse candidates under  $J(n)$ 
24: Evaluate  $J(n)$  and feature vector  $x(n)$  for all coarse and refined candidates
25: Rank-normalize each feature inside the candidate set
26: Compute final tuning score  $T(n)$ 
27: Return directions sorted by  $T(n)$  with angular non-maximum suppression

```

구현에서는 빌드플레이트 항을 텐서에 누적하는 대신 스칼라 B 로 따로 합산하여 $\tilde{R} = R + B$ 를 동등하게 계산한다 (§5.1의 동치, 10^{-12} 이내 수치 확인). 위 의사코드는 그 스칼라 누적과 수학적으로 같은 통합 텐서 형태를 명시한 것이다. 면-면 부분텐서 F 는 $\tilde{F}$ 에서 접지 노드 항을 제외한 부분으로, 특이값 feature 계산에만 쓰인다.

10 비교 대상과 평가 프로토콜

이하 §13까지의 정량 분석에서 사용하는 비교 대상과 평가 절차를 먼저 정의한다. 비교 대상은 다음과 같다.

1. PCA 기반 방향
2. 기존 Support Tensor 고유벡터 방향
3. 초기 eigenvector 기반 concave 방향
4. 본 연구의 Support Flow Tensor Field 방향
5. TOMO_INT3 또는 상용 슬라이서 기반 v_{ss} 최적 방향

각 메쉬에 대해 1° 간격, 임계각 60° 조건의 v_{ss} 등고선을 생성하고, 본 방법의 상위 후보 방향에서의 v_{ss} 값이 전역 최적값에 얼마나 가까운지 평가한다. 방향 좌표가 정확히 일치하지 않더라도 v_{ss} 값이 유사하면 동일한 수준의 최적 배향으로 간주한다. 또한 전역 최적점이 여러 basin에 분산되어 나타나는 경우를 고려하여, 단순 top- k grid point가 아니라 각도상 분리된 top- k basin 대표점을 비교한다. SFTF가 제안한 후보 basin은 TOMO_INT3에 angle step 0° 의 단일 초기 배향으로 다시 입력하여 Mss를 직접 측정하고, 그 측정값을 기준으로 measured optimal 및 measured worst basin을 표시한다.

11 저장된 TOMO_INT3 결과와의 비교

현재 구현을 검증하기 위하여 MeshData에 저장된 `*tomo_int3_vss_grid_1deg_60deg.npz` 파일을 사용하였다. 각 파일은 yaw-pitch 1° grid에서 계산된 TOMO_INT3 support volume v_{ss} , 그리고 해당 grid의 전역 최소 방향을 포함한다. 비교 과정에서는 TOMO grid를 재계산하지 않고, 현재 Support Flow Tensor Field가 제안한 상위 방향을 TOMO grid에 사상한 뒤, 그 위치의 v_{ss} 값을 읽어 전역 최적값과 비교하였다.

SFTF 방향 n 에 대응하는 TOMO yaw-pitch는 TOMO의 rotation matrix $R(yaw, pitch, 0)$ 에 대해

$$R(yaw, pitch, 0)n \approx e_z \quad (32)$$

를 가장 잘 만족하는 grid point로 선택하였다. 방향 벡터는 부호 반전이 같은 build axis를 나타낼 수 있으므로, n 과 $-n$ 을 모두 평가하고 더 작은 v_{ss} 값을 signed result로 사용하였다.

Table 1: 저장된 TOMO_INT3 v_{ss} grid와 현재 SFTF 후보의 비교. Ratio는 TOMO 전역 최적 v_{ss} 대비 해당 SFTF 후보의 v_{ss} 비율이다.

Mesh	TOMO best	SFTF score1	Ratio	SFTF best-of-3	Ratio
Bunny 69k	3201.39	4665.85	1.46	4665.85	1.46
manikin	4657.72	8675.96	1.86	8675.96	1.86
dragon 100k	24446.98	45804.41	1.87	39547.57	1.62
happy 50k	2112.47	25467.94	12.06	23727.49	11.23
lucy 50k	5036.14	14192.68	2.82	14192.68	2.82

Table 1에서 볼 수 있듯이, rank tuning을 적용한 SFTF는 Bunny, manikin, dragon에서는 기존 raw score보다 TOMO optimum에 가까운 후보를 선택하였다. 특히 Bunny의 signed score1 ratio는 약 3.03에서 1.46으로, manikin은 약 3.99에서 1.86으로, dragon의 best-of-3 signed ratio는 약 2.01에서 1.62로 개선되었다. 반면 happy와 lucy에서는 개선이 제한적이었다.

다만 Table 1의 rank tuning 결과는 가중치를 보정한 바로 그 메쉬 집합에서 측정된 in-sample 값이다. 따라서 이 수치는 일반화 성능이라기보다 calibration set에 대한 적합도이며, 가중치의 과적합 여부와 보지 못한 메쉬에 대한 성능은 §12.3의 교차검증으로 별도로 평가한다. 또한 각 feature가 실제 v_{ss} 를 얼마나 예측하는지, 그리고 비대칭 텐서 항이 결과에 실제로 기여하는지는 §12.1에서 분석한다.

11.1 Top- k 점이 아닌 Top- k Basin 기반 최적 배향 표시

TOMO_INT3의 yaw-pitch 등고선에서는 전역 최적값이 하나의 고립된 점이 아니라, 서로 떨어진 여러 basin에서 거의 동일한 값으로 나타날 수 있다. 예를 들어 한 메쉬에서는 $(yaw, pitch) = (69^\circ, 130^\circ)$ 부근과 $(265^\circ, 49^\circ)$ 부근이 모두 전역 최적 또는 near-global optimum에 해당하였다. 이 경우 단순히 v_{ss} 값이 작은 grid point 상위 3개를 표시하면, 동일한 basin 내부의 이웃 grid point만 반복적으로 선택되어 사용자에게 “다른 좋은 배향군도 존재한다”는 정보를 전달하지 못한다.

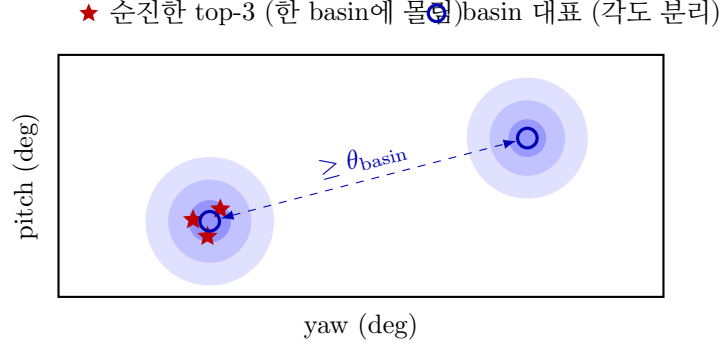


Figure 5: Top- k 점 대 Top- k basin. yaw-pitch 비용 지형에 서로 떨어진 저비용 basin이 여럿 있을 때(파란 영역), 단순히 v_{ss} 가 작은 상위 점을 고르면(빨간 별) 한 basin 안의 이웃 점들만 반복 선택된다. 각도 비최대억제(NMS)로 이미 고른 대표와 θ_{basin} 이상 떨어진 후보만 남기면(파란 원), 서로 다른 near-optimal 배향군을 대표로 제시한다.

따라서 현재 시각화 및 후처리에서는 상위 점(top- k points)이 아니라 상위 basin(top- k basins)을 표시한다. yaw-pitch 점 (ψ, ϕ) 를 build direction $n(\psi, \phi)$ 로 변환한 뒤, 두 배향 사이의 거리를

$$d(n_a, n_b) = \cos^{-1}(n_a \cdot n_b) \quad (33)$$

로 정의한다. 여기서 $|n_a \cdot n_b|$ 가 아니라 $n_a \cdot n_b$ 를 사용한다. 이는 TOMO.INT3의 yaw-pitch 배향에서는 n 과 $-n$ 이 서로 다른 빌드 배향을 나타낼 수 있기 때문이다. 정렬된 후보 목록을 순서대로 훑으면서, 이미 선택된 basin 대표점과의 각도 차이가

$$d(n, n_r) < \theta_{\text{basin}} \quad (34)$$

인 후보는 같은 basin으로 간주하고 제거한다. 현재 구현에서는 $\theta_{\text{basin}} = 12^\circ$ 를 사용하였다.

TOMO.INT3 등고선 자체에 대해서는 v_{ss} grid를 오름차순 또는 내림차순으로 정렬한 뒤, 위의 angular non-maximum suppression¹²을 적용하여 optimal basin 대표점과 worst basin 대표점을 각각 3개씩 선택한다. 이 방식은 전역 최적값이 여러 곳에 존재할 때, 동일 basin의 중복 grid point 대신 서로 다른 최적 배향군을 보여준다.

SFTF 후보에 대해서도 동일한 basin 관점을 적용한다. 먼저 최종 tuning score $T(n)$ 가 낮은 후보와 높은 후보를 각각 정렬하되, 동일 basin 중복을 제거하여 low-score basin 후보와 high-score basin 후보를 각각 선택한다. 현재 구현에서는 각 방향에서 최대 40개의 basin 후보를 유지한다. 그 다음 각 후보를 TOMO.INT3의 초기 yaw-pitch 배향으로 넣고, angle step을 0으로 설정하여 단일 배향만 계산한다. 즉 후보 $q_k = (\text{yaw}_k, \text{pitch}_k)$ 에 대해

$$\text{Mss}_k = \text{TOMO.INT3}(q_k; \Delta\theta = 0) \quad (35)$$

를 직접 측정한다. 마지막으로 이 Mss_k 값으로 후보 basin들을 다시 정렬하고, 측정된 Mss 가 작은 3개를 measured optimal basin, 큰 3개를 measured worst basin으로 표시한다.

이 절차는 SFTF score의 순서를 그대로 최종 optimal/worst로 해석하지 않는다. 대신 SFTF는 넓은 후보 basin을 빠르게 제안하는 역할을 하고, TOMO.INT3는 소수 후보에 대한 정확한 단일 배향 검증기로 사용된다. 따라서 사용자는 하나의 최적 배향만이 아니라, 서로 떨어져 있으면서도 실제 support mass가 작은 여러 배향군을 확인할 수 있다. 이는 출력 방향 최적화에서 순위 1위의 좌표 자체보다 “어떤 선택 가능한 최적 배향군들이 존재하는가”가 더 중요한 경우에 특히 유용하다.

추가 분석에서는 모든 coarse/refined 후보를 TOMO grid에 사상하여 후보 pool 내부의 최저 v_{ss} 를 확인하였다. 기본 512개 coarse 방향에서는 happy 50k의 후보 pool 자체가 TOMO optimum 근방을 충분히 포함하지 못했다. coarse 방향 수를 2048개로 증가시키면 happy 50k에서도 후보

¹²비최대 억제(NMS): 점수가 좋은 후보부터 차례로 채택하되, 이미 채택한 후보와 너무 가까운 후보는 제거해 중복을 없애는 기법(객체 검출에서 유래). 여기서는 “가까움”을 두 배향 사이의 각도로 정의하여, 같은 basin의 이웃 방향이 중복 선택되는 것을 막는다.

pool 내부 최저 ratio가 약 1.91까지 낮아졌다. 이는 happy 50k의 실패가 단순 score tuning 문제만이 아니라, 방향 샘플링 해상도와 후보 생성 방식의 한계도 포함함을 의미한다. 그러나 2048개 후보 pool에서는 현재 feature만으로 좋은 후보를 안정적으로 선택하는 데 어려움이 있었다. 좋은 후보는 대체로 build-plate penalty가 낮으면서도, pair flow와 hit count가 너무 낮지 않은 중간 영역에 위치하였다. 따라서 향후에는 단순 선형 rank score보다, 면-면 self-support coverage 또는 TOMO식 support-volume proxy를 직접 반영하는 feature가 필요하다.

본 실험에서 확인된 주요 개선 요인은 다음과 같다.

- 오버행 법선의 부호와 build direction sign이 직접 반영되었다.
- 면-면 내부 가시성을 ray casting으로 평가하였다.
- 유효 receiver face 조건을 두어 실제 지지 가능한 면만 고려하였다.
- 빌드플레이트까지 직접 지지되어야 하는 오버행을 별도 penalty로 처리하였다.
- 고유축 3개 대신 구면 샘플링과 국소 재탐색을 사용하였다.
- 최종 후보 선택에서 TOMO.INT3 grid와의 비교를 바탕으로 rank-normalized feature tuning을 적용하였다.
- 최종 시각화에서는 top- k 점이 아니라 각도상 분리된 top- k basin 대표점을 표시하여, 서로 다른 near-optimal 배향군을 사용자에게 제공하였다.

동시에 본 결과는 현재 SFTF가 완전히 TOMO.INT3 optimum을 대체한다기보다, 고비용 yaw-pitch 전역 탐색을 줄이기 위한 후보 생성기 또는 warm-start 방법으로 유용함을 시사한다. 특히 상위 후보를 몇 개 제안한 뒤, 그 주변만 TOMO 또는 slicer로 국소 검증하면 전체 탐색 비용을 크게 줄일 수 있다.

12 방법 분석: Feature 기여도, 교차검증, 기존 방법 비교

본 절에서는 제안 방법의 타당성을 세 가지 관점에서 정량적으로 분석한다. 모든 분석은 저장된 1° TOMO.INT3 v_{ss} grid를 재사용하며 GPU 재계산을 필요로 하지 않고, 공개된 분석 스크립트로 재현 가능하다. 별도 언급이 없으면 best-of-3 ratio는 후보 상위 3개(3° 각도 NMS) 중 TOMO grid에 사상하였을 때 signed v_{ss} 가 가장 작은 값의 TOMO 전역 최적 대비 비율이며, 작을수록 좋다.

12.1 Feature 상관관계와 목적함수 Ablation

먼저 각 feature가 실제 v_{ss} 를 얼마나 예측하는지를, 후보 풀 전체에 대한 feature- v_{ss} Spearman 순위상관¹³으로 평가하였다(Table 2).

Table 2: 후보 풀 전체에 대한 feature와 TOMO v_{ss} 의 Spearman 순위상관. 양의 값이 클수록 해당 feature가 비용으로서 유효하다.

Feature	Bunny	manikin	dragon	happy	lucy	평균
$J = R + P + B$	0.46	0.26	0.80	0.27	0.57	0.47
R (Rayleigh)	0.40	-0.33	-0.57	-0.03	0.14	-0.08
P (pair flow)	-0.04	-0.38	-0.72	0.00	-0.59	-0.34
B (build-plate)	0.50	0.66	0.85	0.29	0.62	0.58
H (hit count)	0.24	-0.53	-0.73	-0.00	-0.71	-0.35
S (nuclear)	-0.02	-0.36	-0.70	-0.05	-0.26	-0.28
σ_1	-0.03	-0.34	-0.66	-0.07	0.07	-0.21

¹³스피어먼 순위 상관 계수: 두 변수를 값 대신 순위로 바꾼 뒤 계산하는 상관으로, +1은 완전한 단조 증가, -1은 완전한 단조 감소 관계를 뜻한다. 선형이 아니어도 “큰 값끼리, 작은 값끼리 모이는지”를 잰다. 참고: [27]

build-plate 항 $B(n)$ 만이 모든 메쉬에서 일관된 양의 상관을 가지며(0.29–0.85, 평균 0.58), 목적 함수 J 전체의 예측력(평균 0.47)보다도 높다. 반면 비대칭 텐서에서 파생된 P , S , σ_1 과 hit count H 는 평균 상관이 음수이며 메쉬마다 부호가 뒤집힌다. 즉 이 항들은 고정된 부호의 비용항으로 일반화되지 않는다.

이를 목적함수 수준에서 다시 확인하기 위하여, 후보를 여러 목적함수 변형으로 정렬한 뒤 best-of-3 ratio를 측정하였다(Table 3).

Table 3: 목적함수 ablation의 best-of-3 ratio(낮을수록 좋음). 제안 목적함수 $J = R + P + B$ 와 $\text{nuclear}/\sigma_1$ 을 포함한 구 목적함수가 메쉬별로 완전히 동일하다.

Ranking variant	Bunny	manikin	dragon	happy	lucy	평균
T (rank tuned, in-sample)	1.46	1.86	1.62	11.23	2.82	3.80
$J = R + P + B$ (제안)	2.47	3.99	2.01	11.15	2.82	4.49
$R+P+B+0.05S+0.05\sigma_1$ (구 목적함수)	2.47	3.99	2.01	11.15	2.82	4.49
B 단독	2.57	2.60	1.94	16.04	2.82	5.19
$0.05S + 0.05\sigma_1$ (텐서 단독)	5.82	5.38	6.57	11.86	5.62	7.05
P 단독	5.82	4.92	6.27	12.07	7.51	7.32

핵심적으로 제안 목적함수 $J = R + P + B$ 와 $\text{nuclear}/\sigma_1$ 항을 포함한 구 목적함수의 결과가 모든 메쉬에서 동일하다. 이는 0.05 가중치의 $\text{nuclear}/\sigma_1$ 항이 최종 후보 선택에 사실상 기여하지 않음을 의미하며, 본 개정판에서 이 항들을 목적함수에서 제외한 근거가 된다. 또한 텐서 파생 항만 사용하는 변형($0.05S + 0.05\sigma_1$, P 단독)은 ratio가 가장 나쁘다. 즉 본 방법의 예측력은 비대칭 텐서 구조가 아니라 build-plate penalty와 내부 가시성 평가에서 비롯되며, 텐서 항은 보조적 역할에 그친다.

12.2 통합 텐서로의 재구성: 빌드플레이트 항의 Rayleigh 동치

§12.1의 ablation은 빌드플레이트 항 B 가 예측력을 거의 전담하고 텐서에서 파생된 Rayleigh 항 R 은 부호가 메쉬마다 뒤집힘을 보였다(Table 2). 그러나 §5.1의 가상 접지 노드 해석에 따르면 B 자체가 증강 흐름 텐서의 Rayleigh 기여이며, 통합 Rayleigh cost는 $\tilde{R} = R + B$ (식 (18))로 정확히 표현된다. 본 절은 그 통합 항이 실제로 더 나은 예측자인지 검증한다.

먼저 동치를 수치로 확인하였다. 저장된 5개 메쉬의 후보 풀 전체에 대해 증강 텐서 $\tilde{F}(n)$ 의 Rayleigh cost를 직접 계산한 결과, 모든 후보에서 $|\tilde{R} - (R + B)| < 10^{-12}$ 로 부동소수점 오차 내에서 동일하였다.

Table 4은 통합 항 $\tilde{R}$ 의 Spearman 상관을 단일 항들과 비교한다. R 만으로는 평균 -0.08 (부호 불안정)이지만, 접지 노드를 더한 $\tilde{R} = R + B$ 는 평균 0.60 으로 단일 feature 중 최고이며, 현재 목적함수 $J = R + P + B$ (평균 0.47)보다도 높다. J 가 $\tilde{R}$ 보다 낮은 이유는 면-면 지지량 P (평균 상관 -0.34 , Table 2)가 음의 신호로 작용하여 합산 시 예측력을 떨어뜨리기 때문이다.

Table 4: 통합 Rayleigh 항 $\tilde{R} = R + B$ 와 단일 항의 TOMO v_{ss} Spearman 상관(높을수록 좋음). $\tilde{R}$ 는 증강 텐서로부터 직접 계산하였으며 $R + B$ 와 10^{-12} 이내에서 일치한다.

Mesh	R	B	$\tilde{R} = R+B$	$J = R+P+B$
Bunny	0.40	0.50	0.52	0.46
manikin	-0.33	0.66	0.65	0.26
dragon	-0.57	0.85	0.89	0.80
happy	-0.03	0.29	0.31	0.27
lucy	0.14	0.62	0.62	0.57
평균	-0.08	0.58	0.60	0.47

이를 best-of-3 ratio로도 확인하였다(Table 5). 통합 항만으로 후보를 정렬하면(즉 P 를 제거), 후보 생성이 정상인 4개 메쉬에서 평균 ratio가 J 의 2.82에서 $\tilde{R}$ 의 2.50으로 개선된다. happy는 후보 풀 자체가 near-optimal을 포함하지 못하는 별도 사례(§12.5)로, 이 메쉬에서는 통합 항이 오히려 악화되나 이는 scoring이 아니라 샘플링 한계에 기인한다.

Table 5: $J = R + P + B$ 와 통합 항 $\tilde{R} = R + B$ 로 후보를 정렬했을 때의 best-of-3 ratio(낮을수록 좋음). happy(후보 생성 실패, §12.5)를 제외한 평균에서 $\tilde{R}$ 가 J 를 앞선다.

Mesh	$J = R+P+B$	$\tilde{R} = R+B$
Bunny	2.47	2.57
manikin	3.99	2.89
dragon	2.01	1.70
lucy	2.82	2.82
평균(happy 제외)	2.82	2.50

종합하면 통합 텐서 재구성은 (i) 빌드플레이트 비용을 흐름 텐서의 한 항으로 정식화하여 “텐서” 표현을 정당화하고, (ii) 음의 신호인 P 를 제거한 단일 텐서 Rayleigh cost $\tilde{R}$ 가 기존 J 보다 일관되게 우수함을 보인다.

높이 규약의 검증. 접지 항은 면-면 항과 부호가 반대인 높이 규약(감쇠 대신 증폭)을 쓴다. 이 비대칭이 임의적이지 않음을 보이기 위해 네 가지 플레이트 규약을 비교하였다. 현재의 증폭 $B_{\text{amp}} = \sum O_i A_i (1 + \alpha h)$, 무높이 $B_{\text{flat}} = \sum O_i A_i$, 면-면과 대칭인 감쇠 $B_{\text{att}} = \sum O_i A_i / (1 + \alpha h)$, 면-면 가중치 법칙을 완전히 따른 $B_{\text{uni}} = \sum O_i^2 A_i / (1 + \alpha h)$ 이다. 평균 Spearman 상관은 각각 0.58, 0.51, 0.43, 0.47로, **현재의 증폭 규약이 가장 높고 가장 일관적이며**, 면-면과 대칭으로 맞춘 감쇠 규약 B_{att} 가 가장 낮았다(happy에서 음의 상관). 즉 “면 자기지지는 약화, 키 큰 빌드플레이트 지지는 강화”라는 부호 비대칭은 물리적으로 옳고 실증적으로 뒷받침된다. 다만 완전 통일 규약 B_{uni} 는 manikin 등 세장형 형상에서 best-of-3가 크게 개선되어 (예: manikin 2.89 $\rightarrow$ 1.37) 향후 형상 적응적 규약의 여지를 남긴다.

12.3 Rank Tuning의 교차검증

Table 1의 rank tuning 가중치는 보고 대상 메쉬에서 함께 보정되었으므로, 일반화 성능을 평가하기 위해 Leave-One-Mesh-Out(LOMO) 교차검증¹⁴을 수행하였다. 각 메쉬를 hold-out으로 두고 나머지 메쉬에서 가중치를 다시 적합한 뒤 (메쉬 내부 rank 정규화 feature에 대한 ridge 회귀¹⁵, 목표는 메쉬 내부 rank 정규화 v_{ss}), hold-out 메쉬의 best-of-3 ratio를 측정하였다. 후보 생성 자체가 실패하는 happy(아래 §12.3 말미 및 oracle 참조)는 제외하고, 후보 풀에 near-optimal 방향이 포함되는 4개 메쉬에 대해 평가하였다.

¹⁴교차검증(cross-validation): 데이터 일부를 학습에서 빼 두었다가 검증에만 사용해, 보지 못한 데이터에 대한 성능을 추정하는 방법. LOMO는 “메쉬 하나씩 빼기” 방식으로, 한 메쉬를 제외하고 나머지로 가중치를 맞춘 뒤 제외한 메쉬에서 성능을 잰다. 참고: [28]

¹⁵능형 회귀(ridge regression): 회귀 가중치가 지나치게 커지지 않도록 L_2 벌점을 더해 과적합을 줄이는 선형 회귀. 참고: [12]

Table 6: Rank tuning의 LOMO 교차검증 best-of-3 ratio(낮을수록 좋음). in-sample 열은 Table 1에서 보고된 보정셋 적합 값이다. oracle은 후보 풀 내부에 존재하는 최선 방향으로, 후보 생성의 상한을 나타낸다.

Mesh	oracle	untuned J	in-sample (보고값)	LOMO
Bunny	1.46	2.47	1.46	2.20
manikin	1.13	3.99	1.86	2.62
dragon	1.29	2.01	1.62	1.86
lucy	1.02	2.82	2.82	2.82
평균	1.22	2.82	1.94	2.37

보고된 in-sample 평균 1.94는 정직한 교차검증에서 재현되지 않으며, hold-out 평균은 2.37로 약 0.43(약 22%) 높다. 이 차이가 가중치 보정의 낙관 편향이다. 또한 7개 가중치 중 fold 간에 안정적으로 재현되는 것은 $R(\approx 0.7)$ 과 $B(\approx 1.0)$ 뿐이며, 보고값의 큰 $P = -2.39$, $H = 2.31$, $S = -2.29$, $\sigma_1 = 1.64$ 는 어느 fold에서도 재현되지 않는다. 실제로 안정적인 feature만 사용하는 축소 모델의 LOMO 평균 ratio는 B 단독 2.48, $R + B$ 2.46, $R + P + B$ 2.44로 전체 7-feature 모델(2.37)과 큰 차이가 없고, 모두 untuned $J(2.82)$ 보다 우수하다. 따라서 7개의 보정된 가중치 대신 R , B (필요 시 P) 중심의 소수 feature 튜닝이 더 단순하면서도 견고하다. 한편 모든 변형에서 hold-out ratio가 oracle(1.22)보다 분명히 크다는 점은, 개선 여지가 후보 생성이 아니라 scoring 단계에 남아 있음을 시사한다.

12.4 기존 방법과의 통합 비교

마지막으로 동일한 best-of-3 ratio 기준에서 PCA 방향, Support Tensor 고유벡터 방향, 그리고 SFTF(untuned 및 LOMO 교차검증 tuned)를 비교하였다(Table 7). PCA¹⁶는 면적 가중 면 중심 공분산의 주축, Support Tensor는 $M = \sum_i A_i m_i m_i^T$ 의 고유벡터를 사용하며, 각 방법의 3개 축($\pm$ 포함)에 대한 최선값을 취하였다. SFTF-tuned 열은 in-sample 값이 아니라 §12.3의 LOMO 값이다.

Table 7: 기존 방법과 SFTF의 통합 비교(best-of-3 signed v_{ss} ratio, 낮을수록 좋음). SFTF-tuned는 교차검증된 일반화 추정치이다.

Mesh	PCA	Support Tensor	SFTF untuned	SFTF tuned (LOMO)	oracle
Bunny	2.82	3.71	2.47	2.20	1.46
manikin	2.65	2.50	3.99	2.62	1.13
dragon	2.69	2.33	2.01	1.86	1.29
lucy	6.37	8.67	2.82	2.82	1.02
평균	3.63	4.30	2.82	2.37	1.22

교차검증된 SFTF(평균 2.37)는 PCA(3.63)보다 약 35%, Support Tensor 고유벡터(4.30)보다 약 45% 우수하며, 튜닝하지 않은 $J(2.82)$ 조차 두 기존 방법보다 낫다. 특히 Support Tensor 고유벡터는 PCA보다도 나쁜데(lucy에서 8.67), 이는 법선 분포의 통계적 주축만으로는 방향 의존적 지지구조량을 설명하지 못한다는 본 연구의 출발 가정을 정량적으로 뒷받침한다. 이 비교는 보고된 in-sample 수치를 버리고 보수적인 교차검증 수치를 사용하여도 SFTF의 기존 방법 대비 우위가 유지됨을 보여준다.

12.5 추가 강건성 분석: 임계각, 샘플링 밀도, 차원 정규화

임계각 정합. SFTF의 오버행 정의 $O_i(n) = \max(0, -m_i \cdot n)$ 는 임계각과 무관하게 모든 하향면을 포함하지만, TOMO-INT3는 하향면($m_i \cdot n < 0$) 중 $-m_i \cdot n > \cos \theta_c$ 인 면만 지지 필요로 간주한다.

¹⁶주성분 분석(principal component analysis): 데이터가 가장 넓게 퍼진 방향(주축)을 공분산 행렬의 고유벡터로 찾는 기법. 여기서는 면 중심들의 분포 주축을 출력 방향 후보로 쓴다. 참고: [15]

동일 게이트를 SFTF 오버행에 적용하여 정의를 정합시킨 결과, $\theta_c = 60^\circ$ (TOMO와 동일) 설정에서 LOMO 평균 best-of-3 ratio가 2.37에서 2.82로 오히려 악화되었고, $\theta_c = 45^\circ$ 에서 2.42로 baseline과 유사하였다. 즉 임계각 게이트 정합은 일관된 개선을 주지 않으며, 이는 rank tuning이 오버행 정의의 차이를 이미 흡수하기 때문으로 해석된다. 따라서 기본 구현은 게이트를 적용하지 않는다.

샘플링 밀도와 후보 생성 한계. coarse Fibonacci sphere 방향 수를 늘리며 후보 풀 내부 최선값 (oracle ratio)을 측정하였다(Table 8).

Table 8: coarse 방향 수에 따른 oracle(후보 풀 내부 최선) ratio. happy를 제외한 메쉬는 512개에서 이미 near-optimal 방향을 포함한다.

coarse 방향 수	Bunny	manikin	dragon	happy	lucy
512	1.46	1.13	1.29	4.35	1.02
1024	1.50	1.11	1.33	3.94	1.02
2048	1.61	1.06	1.28	1.91	1.04
4096	1.62	1.13	1.27	1.91	1.30

happy의 oracle ratio는 512개에서 4.35로, near-optimal 방향이 후보 풀에 포함되지 않는다. 방향 수를 2048개로 늘리면 1.91까지 낮아져 후보 생성 한계가 부분적으로 해소된다. 반면 다른 메쉬는 512개에서 이미 oracle이 1.0–1.5로 충분하며, 방향 수 증가의 이득이 없다. 다만 happy는 후보가 보장되어도 untuned J 의 best-of-3 ratio가 4096개에서조차 10.51에 머물러, 샘플링과 scoring 양쪽 모두에서 한계를 보인다. 이는 happy를 별도의 난해 사례로 분리하는 근거가 된다.

목적함수 차원 정규화. 목적함수 $J = R + P + B$ 의 항들은 단위가 상이하다($R, P \sim$ 면적², $B \sim$ 면적). 세 항을 후보 풀 내부에서 동일 척도로 정규화한 뒤 등가중 합산하면, raw J 의 Spearman 상관이 0.52에서 min-max 정규화¹⁷ 시 0.14, z-score 정규화¹⁸ 시 0.10으로 크게 하락하며, best-of-3 ratio도 2.82에서 각각 3.53, 3.86으로 악화된다. 이는 raw J 의 예측력이 여러 항의 균형이 아니라 가장 큰 척도를 갖는 build-plate 항의 지배에서 비롯됨을 보여준다. 따라서 가중치의 물리적 의미는 단순 차원 정규화가 아니라 비예측 항을 제거하고 R, B 중심으로 축소함으로써 (§12.3) 확보하는 것이 타당하다.

13 통제된 5-그룹 형상에 대한 네 구현 대비 검증

앞 절들이 저장된 Stanford 메쉬 grid에 대한 정확도와 일반화를 다루었다면, 본 절은 난이도를 통제된 형상 집합에서 SFTF의 알고리즘적 특성을 TOMO_INT3, TOMO_CUDA, Python SFTF, SFTF_C++ 네 구현으로 직접 대비한다. 검증은 `five_group_verification.py`로 일괄 수행하였으며, SFTF는 입력 메쉬만으로 후보 풀을 생성하고(coarse 512방향, 후보 풀 1292개), TOMO_INT3와 TOMO_CUDA는 동일 메쉬에 대해 yaw-pitch 3° grid ($120 \times 120 = 14,400$ 방향), 임계각 60° 의 전수 sweep을 DLL로 수행하였다.¹⁹ SFTF_C++는 Python SFTF와 같은 후보 생성 알고리즘을 C++ DLL로 포팅한 구현이며, 측정값은 순수 계산 시간(`sftf_compute_directions`)이다. 최신 검증 표(`_5G_CA60deg_4Method.table.html`)는 A-E 전체 35개 메쉬에 대해 네 구현의 시간이 모두 채워진 상태이다.

13.1 실험 설정

데이터셋은 형상 난이도와 메쉬 규모라는 두 축을 의도적으로 분리해 통제된 다섯 그룹으로 구성한다. A-C는 규모를 $\sim 10^1$ – 10^5 면 범위에 두고 난이도 축을 따라 기본 도형에서 freeform 유기

¹⁷ 최솟값을 0, 최댓값을 1로 선형 사상하는 특징 스케일링(정규화). 참고: [11]

¹⁸ 표준 점수. 평균을 빼고 표준편차로 나눠 “평균 0, 분산 1”로 만드는 정규화. 참고: [11]

¹⁹ 36개 메쉬를 임계각 5종($\theta_c \in \{0, 30, 45, 60, 90\}^\circ$)에 대해 두 backend로 sweep하는 대규모 배치를 다루기 위해, 본 검증의 sweep 격자는 선행 GPU-MSST의 1° ($360 \times 360 = 129,600$ 방향)보다 성긴 3° 로 두었다. 더 조밀한 1° grid에서는 TOMO sweep 시간이 약 9배로 늘어 SFTF 대비 속도 격차가 더 벌어진다.

형상까지 변화시켜, 전수 sweep의 고정비용 바닥(메쉬 복잡도와 거의 무관한 비용)을 드러낸다. D-E는 난이도를 실제 제조 부품으로 고정한 채 규모 축을 256면에서 1.04×10^6 면까지 밀어, SFTF 후보 평가 비용이 메쉬 복잡도에 적응하여 증가하는 적응비용을 드러낸다. 따라서 A-E는 단일 크기 램프가 아니며, 그룹 C와 D의 면수 구간이 일부 겹치는 것은 두 축을 분리해 통제된 설계의 결과이지 순서 오류가 아니다.

- A (기본 도형, 5개; 난이도 하단): cube, sphere, cylinder, cone, torus (면수 $\sim 10^1$ – 10^3).
- B (단순 기능 부품, 5개; 난이도 중단): u_bracket, hook, c_clamp, pipe_elbow, hollow_box.
- C (유기적/freeform 벤치마크, 5개; 난이도 상단): Bunny(69,662면), manikin, dragon($\sim 10^5$ 면), happy, lucy 등 표준 스캔/조각 메쉬. 앞 절들의 Stanford grid 메쉬가 곧 이 그룹이다.
- D (Thingi10K set 1, 10개; 규모 하단): 면수 256–88,106의 실제 제조 부품 [31].
- E (Thingi10K set 2, 10개; 규모 상단): 면수 약 5.1×10^5 – 1.04×10^6 의 대형 고해상도 메쉬.

A-E의 35개 메쉬에 대해 Python SFTF 후보 평가 시간, TOMO_INT3 DLL sweep 시간, TOMO_CUDA DLL sweep 시간, SFTF_C++ 후보 평가 시간, SFTF 최상위 방향의 hit 수와 후보 점수, 그리고 TOMO contour HTML을 기록하였다. 이 절은 방향 정확도 ratio가 아니라 계산 구조와 구현 축의 대비를 다루며, 방향 정확도의 정량 분석은 §12.1–§12.4의 grid 비교가 담당한다.

13.2 전체 계산 시간 대비

Table 9은 35개 메쉬의 네 구현 측정값을 요약한다. 가장 두드러진 특성은 고정비용 대 적응비용의 대조이다. TOMO_INT3는 면이 12개뿐인 cube에도 약 4초가 소요되어, yaw-pitch 전수 sweep 특성상 메쉬 복잡도와 거의 무관한 큰 고정비용을 갖는다(3° 격자에서도 14,400개 방향을 모두 평가하기 때문이다). 반면 SFTF의 비용은 메쉬 복잡도에 적응하여 기본 도형에서는 0.12–0.74초, 유기적 벤치마크(C)에서는 1.0–3.2초, Thingi10K 부품(D)에서는 0.36–3.42초, 50만–100만 면 대형 메쉬(E)에서도 9.71–22.11초에 그친다. 전체 35개 기준으로 Python SFTF는 TOMO_INT3(3°) 대비 1.4–34× 빠르며, 평균 속도비는 약 9×이다. SFTF_C++는 추가로 Python SFTF 대비 3.9–233× 빠르고, TOMO_INT3(3°) 대비 10–5,160× 빠르다. 즉 SFTF는 고비용 전역 sweep을 대체하는 것이 아니라, 그 앞단에서 유망 배향을 수 배에서 수천 배 빠르게 제안하는 후보 생성기로 적합하며, sweep 격자를 1° 로 조밀하게 할수록 이 격차는 더 벌어진다.

TOMO_CUDA의 결과는 더 미묘하다. A/B의 단순 형상에서는 TOMO_CUDA가 TOMO_INT3보다 대체로 빠르며 전수 sweep의 GPU 가속 효과를 보인다. 그러나 C/D/E의 유기적 메쉬와 대형 메쉬에서는 TOMO_CUDA가 TOMO_INT3보다 느린 경우가 많고, 전체 평균으로는 TOMO_CUDA/TOMO_INT3 시간비가 약 1.07이다. 이는 본 검증 환경에서는 GPU backend 자체보다 전수 sweep의 입력 준비, 메쉬 복잡도, DLL 내부 자료구조 및 메모리 이동 비용이 성능을 좌우할 수 있음을 뜻한다. 따라서 본 논문의 핵심 속도 이득은 “INT3를 CUDA로 바꾸는 하드웨어 축”보다 “ 3° 격자 기준 14,400개 (1° 에서는 129,600개) 전수 방향을 1292개 후보 평가로 줄이는 알고리즘 축”에서 나온다.

Table 9: 5-그룹 35개 메시에 대한 네 구현의 계산 시간. t_{INT3} 와 t_{CUDA} 는 각각 TOMO_INT3/TOMO_CUDA 3° 전수 sweep($\theta_c = 60^\circ$) DLL 시간, t_{py} 는 Python SFTF 후보 평가, $t_{\text{C++}}$ 는 SFTF_C++ 순수 계산 시간이다. hit은 Python SFTF 최상위 방향의 유효 면-면 지지 수이다. Speed columns는 각각 $t_{\text{INT3}}/t_{\text{py}}$ 와 $t_{\text{INT3}}/t_{\text{C++}}$ 이다.

ID	형상	면수	hit	$t_{\text{INT3}}(\text{s})$	$t_{\text{CUDA}}(\text{s})$	$t_{\text{py}}(\text{s})$	$t_{\text{C++}}(\text{ms})$	INT3/py	INT3/C++
<i>A: 기본 도형</i>									
A1	cube	12	0	4.12	1.26	0.13	4.1	32×	1,006×
A2	sphere	2,976	498	4.29	1.53	0.74	23.9	6×	179×
A3	cylinder	192	0	4.12	2.16	0.17	1.3	24×	3,172×
A4	cone	96	0	4.13	1.47	0.12	0.8	34×	5,157×
A5	torus	2,304	184	4.28	1.48	0.45	14.2	10×	301×
<i>B: 단순 부품</i>									
B1	u_bracket	28	2	4.13	1.35	0.15	3.9	28×	1,059×
B2	hook	1,560	235	4.21	1.44	0.58	15.8	7×	266×
B3	c_clamp	28	1	4.13	1.63	0.17	0.8	24×	5,157×
B4	pipe_elbow	2,400	500	4.27	1.49	0.81	25.3	5×	169×
B5	hollow_box	24	4	4.13	1.64	0.21	0.9	20×	4,592×
<i>C: 유기적 벤치마크</i>									
C1	Bunny	69,662	81	6.17	6.97	1.69	247.7	4×	25×
C2	manikin	13,672	134	4.38	3.70	1.02	44.3	4×	99×
C3	dragon	99,999	970	9.65	12.11	3.17	323.3	3×	30×
C4	happy	49,944	240	5.92	6.63	1.50	130.1	4×	45×
C5	lucy	49,999	542	6.44	8.87	1.87	172.5	3×	37×
<i>D: Thingi10K set 1</i>									
D1	37009	23,909	57	5.76	13.28	1.37	114.5	4×	50×
D2	37010	13,247	35	4.95	13.28	1.24	63.5	4×	78×
D3	37011	17,288	82	5.06	5.81	1.15	89.2	4×	57×
D4	37091	256	42	4.13	2.00	0.36	5.2	11×	795×
D5	37095	468	65	4.17	3.62	0.49	6.2	9×	672×
D6	37111	88,106	386	12.33	21.37	3.42	465.4	4×	26×
D7	37137	64,078	434	10.18	22.16	2.90	305.5	4×	33×
D8	37402	8,322	119	5.00	6.82	2.43	49.7	2×	101×
D9	37415	13,340	77	5.73	11.52	2.96	65.4	2×	88×
D10	37416	12,036	496	4.74	6.10	3.39	80.4	1.4×	59×
<i>E: Thingi10K set 2, 대형 메시</i>									
E1	39507	969,520	562	129.39	131.73	16.92	4387.6	8×	29×
E2	45809	517,589	107	64.89	73.88	9.71	1984.5	7×	33×
E3	45811	1,037,876	1,309	132.74	145.57	18.69	4515.6	7×	29×
E4	46012	638,754	50	82.74	101.35	11.15	2583.8	7×	32×
E5	55280	900,490	1,072	118.42	138.23	16.78	3957.9	7×	30×
E6	59197	786,432	699	99.04	105.32	17.50	4119.5	6×	24×
E7	61384	516,608	1,766	65.83	80.09	12.85	2065.0	5×	32×
E8	64445	514,938	2,288	21.98	31.69	15.05	2208.0	1.5×	10×
E9	64446	514,938	2,288	21.89	31.78	14.93	2211.4	1.5×	10×
E10	65942	754,688	1,175	108.07	111.99	22.11	3201.6	5×	34×

13.3 그룹별 평균과 구현 축의 분리

Table 10: 그룹별 평균 실행 시간과 속도비. SFTF_C++ 평균은 ms 단위이며, 나머지는 초 단위이다. TOMO_CUDA/TOMO_INT3 비율이 1보다 크면 CUDA backend가 INT3보다 느린 경우이다.

Group	py (s)	INT3 (s)	CUDA (s)	C++ (ms)	INT3/py	CUDA/INT3
A	0.32	4.19	1.58	8.9	6–34×	0.31–0.52
B	0.38	4.17	1.51	9.3	5–28×	0.33–0.40
C	1.85	6.51	7.66	183.6	3–4×	0.84–1.38
D	2.17	6.21	10.60	124.5	1.4–11×	0.48–2.68
E	15.57	84.50	95.16	3123.5	1.5–8×	1.02–1.45

13.4 대형 메쉬로의 확장성(그룹 E)

그룹 E는 면수 약 5.1×10^5 에서 1.04×10^6 에 이르는 Thingi10K 대형 메쉬 10개로, SFTF가 실제 부품 규모를 넘어 고해상도 메쉬에서도 동작하는지를 본다. 핵심 관찰은 *SFTF 비용의 완만한 증가*이다. 면수가 D6(88,106면)의 약 6–12배인 E 메쉬에서도 Python SFTF 후보 평가는 9.71–22.11초로, 면수 증가에 대해 대략 선형 이하로만 늘어난다. 이는 ray 예산을 $K_{ray} = \text{clip}(0.02N_f, 500, 3000)$ 으로 상한 3000개에 고정하였기 때문으로, 큰 메쉬에서는 면 전처리와 BVH 구성이 비용을 지배하되 ray casting 비용은 포화한다. SFTF_C++는 같은 대형 메쉬에서 1.98–4.52초에 후보 평가를 끝내며, Python 대비 3.9–6.9× 빠르다. 이 두 후보 생성기 비용은 sweep 격자와 무관하다(SFTF는 전수 sweep을 하지 않는다).

반면 TOMO_INT3의 3° 전수 sweep은 E 그룹에서 22–133초가 걸린다(1°에서는 약 9배인 126–1,077초). E8/E9처럼 동일한 514,938면 메쉬에서 약 22초로 비교적 짧은 경우도 있지만, 대부분의 대형 메쉬에서는 수십 초에서 백 초 이상의 시간이 필요하다. TOMO_CUDA 역시 32–146초로, 이 검증 환경에서는 대형 메쉬에서 INT3보다 빠르지 않았다. 이는 전수 sweep backend의 하드웨어 가속보다 후보 수를 줄이는 알고리즘적 축소가 대형 메쉬에서도 더 안정적인 속도 이득을 제공함을 보여준다.

내부 일관성도 확인된다. E8(64445)과 E9(64446)은 사실상 동일한 메쉬(514,938면)로, SFTF는 동일한 최상위 점수(−0.9824), 동일한 방향, 동일한 hit(2,288)을 산출하여 결정론적 재현성을 보였고 차이는 실행 시간(15.05 대 14.93초)뿐이었다. 또한 유효 면-면 지지 수(hit)는 메쉬 디테일이 풍부해질수록 증가하여 (예: E7 1,766, E8/E9 2,288) 고해상도 형상의 내부 자기지지 구조가 SFTF의 흐름 텐서에 충실히 반영됨을 시사한다.

13.5 방향별 비용 구조 분해

TOMO_INT3가 각 배향을 단일 스칼라 v_{ss} 로 요약하는 것과 달리, SFTF는 최상위 방향의 비용을 Rayleigh 항 R , 면-면 지지량 P , 빌드플레이트 penalty B , 그리고 유효 면-면 지지 수 hit으로 분해하여 제공한다 (Table 11). 이 분해는 “왜 그 방향이 좋은가”에 대한 해석을 직접 제공한다. 점에서 SFTF의 구별되는 특성이다.

블록 기본 도형인 cube, cylinder, cone은 hit=0이고 $R = P = 0$ 이며 비용이 전적으로 빌드플레이트 항 B 로만 구성된다. 이는 “자기지지 관계가 없고 평면을 바닥에 안착시키면 지지가 불필요”함을 알고리즘이 직접 식별함을 뜻한다. 반대로 sphere, torus 및 모든 유기적/실제 부품은 hit>0로 면-면 자기지지 구조가 존재하며, R, P, B 가 함께 기여한다. 예컨대 D10(37416)은 hit=496, $P \approx 1.2 \times 10^4$ 로 면-면 자기지지 구조가 지배적이고 $B \approx 1.3$ 으로 빌드플레이트 의존이 작은 반면, D6(37111)은 $B \approx 1.6 \times 10^3$ 로 빌드플레이트 지지 요구가 크다. 유기적 메쉬에서도 같은 대비가 나타나, dragon은 hit=970에 R, P, B 가 고르게 분포하는 반면 Bunny는 $B \approx 176$ 이 지배적이다. 이러한 regime 구분은 단일 v_{ss} 값만으로는 드러나지 않는다.

Table 11: 대표 메쉬의 SFTF 최상위 방향 비용 분해. 볼록 기본 도형(cube, cone)은 $\text{hit} = 0$, $R = P = 0$ 로 순수 빌드플레이트 지지이며, 나머지는 면-면 자기지지(P)와 빌드플레이트 지지(B)가 함께 나타난다.

ID	형상	hit	R	P	B
A1	cube	0	0	0	401.5
A4	cone	0	0	0	179.8
A2	sphere	498	47.3	109.5	0.02
A5	torus	184	15.9	40.3	209.1
B2	hook	235	19.9	55.4	128.7
C1	Bunny	81	1.4	25.2	176.3
C3	dragon	970	109.4	267.6	417.5
D6	37111	386	227.9	932.8	1,613
D10	37416	496	6,381	12,190	1.3

13.6 임계각에 따른 지지체적과 최적 배향 변화

같은 형상이라도 출력 가능한 임계각 θ_c 에 따라 어떤 면이 오버행으로 분류되는지가 달라지므로, TOMO 지지체적과 그 전역 최적 배향도 θ_c 에 의존한다. 이를 정량화하기 위해 35개 메쉬 전부를 $\theta_c \in \{0, 30, 45, 60, 90\}^\circ$ 의 다섯 임계각에 대해 TOMO_INT3와 TOMO_CUDA로 다시 sweep하였다(3° 격자). 각 임계각마다 네 구현을 한 페이지에 모은 표(`.5G_CA $\langle\theta_c\rangle$ deg_4Method_table.html`)와 다섯 그룹의 best/worst 배향을 한 프레임에 담은 Polyscope 오버레이(`.5G_CA $\langle\theta_c\rangle$ deg_best_worst_polyscope.png`, 그림 6)를 생성하였다.

Table 12은 대표 메쉬의 TOMO_INT3 전역 최소 v_{ss} 가 θ_c 증가에 따라 단조 감소함을 보인다. 이는 직관과 일치한다. θ_c 가 커질수록 더 가파른 면까지 자기지지로 허용되어 지지 요구가 줄기 때문이다. 감소의 양상은 형상마다 다르다. cube는 $\theta_c \leq 30^\circ$ 에서는 모서리를 세운 배향이 최적이고 $v_{ss} > 0$ 이지만 $\theta_c \geq 45^\circ$ 에서는 평면을 바닥에 안착시켜 $v_{ss} = 0$ 이 되는 문턱 거동을 보이며, hook은 $1,736 \rightarrow 0$ 으로 매끄럽게 감소한다. 대형/유기적 메쉬에서는 전역 최적 배향 자체가 이동하여, 예컨대 37009의 최적 ($yaw, pitch$)는 $\theta_c \leq 45^\circ$ 의 ($\approx 345^\circ, 0^\circ$) 부근에서 $\theta_c \geq 60^\circ$ 에는 다른 basin인 ($\approx 150^\circ, 186^\circ$) 부근으로 도약한다.

Table 12: 임계각별 TOMO_INT3 전역 최소 v_{ss} (대표 메쉬, 3° 격자). θ_c 증가에 따라 지지체적이 단조 감소한다. θ_c 가 큰 영역에서는 거의 모든 면이 자기지지가 되어 지지가 사실상 불필요하며, 이때 MSST 분해 $V_{ss} = V_{tc} - V_o - V_{nv}$ 의 수치 잔차로 v_{ss} 가 작은 음수로 나타날 수 있다(실질적으로 0).

ID	메쉬	$\theta_c=0^\circ$	30°	45°	60°	90°
A1	cube	1,769	1,769	0	0	0
B2	hook	1,736	725	319	133	0
C1	Bunny	49,780	23,660	13,970	5,147	-7,857
C3	dragon	217,000	119,400	84,500	24,450	-18,090
D1	37009	65,810	4,729	1,061	-941	-7,226
D6	37111	99,200	58,980	38,740	9,703	-6,270
E1	39507	26,380	12,510	6,493	-6,337	-13,410
E7	61384	75,040	21,140	17,590	-3,503	-14,610

주의할 대비는, 위에서 TOMO 최적 배향이 θ_c 에 따라 이동하는 반면 현재 SFTF 후보는 θ_c 에 불변이라는 점이다. 이는 SFTF 후보 생성기가 오버행 계수 $O_i(n) = \max(0, -m_i \cdot n)$ 를 임계각으로 절단하지 않는 설정(USE_CRITICAL_ANGLE=False)으로 구현되어 있기 때문으로, 그 결과 다섯 임계각의 best/worst 오버레이(그림 6)는 시각적으로 동일하다. 따라서 그림 6은 임의의 한 임계각($\theta_c = 60^\circ$) 결과를 대표로 보인 것이며, θ_c 의존 후보 생성(오버행 절단을 켜 SFTF)은 향후 과제로 남는다.

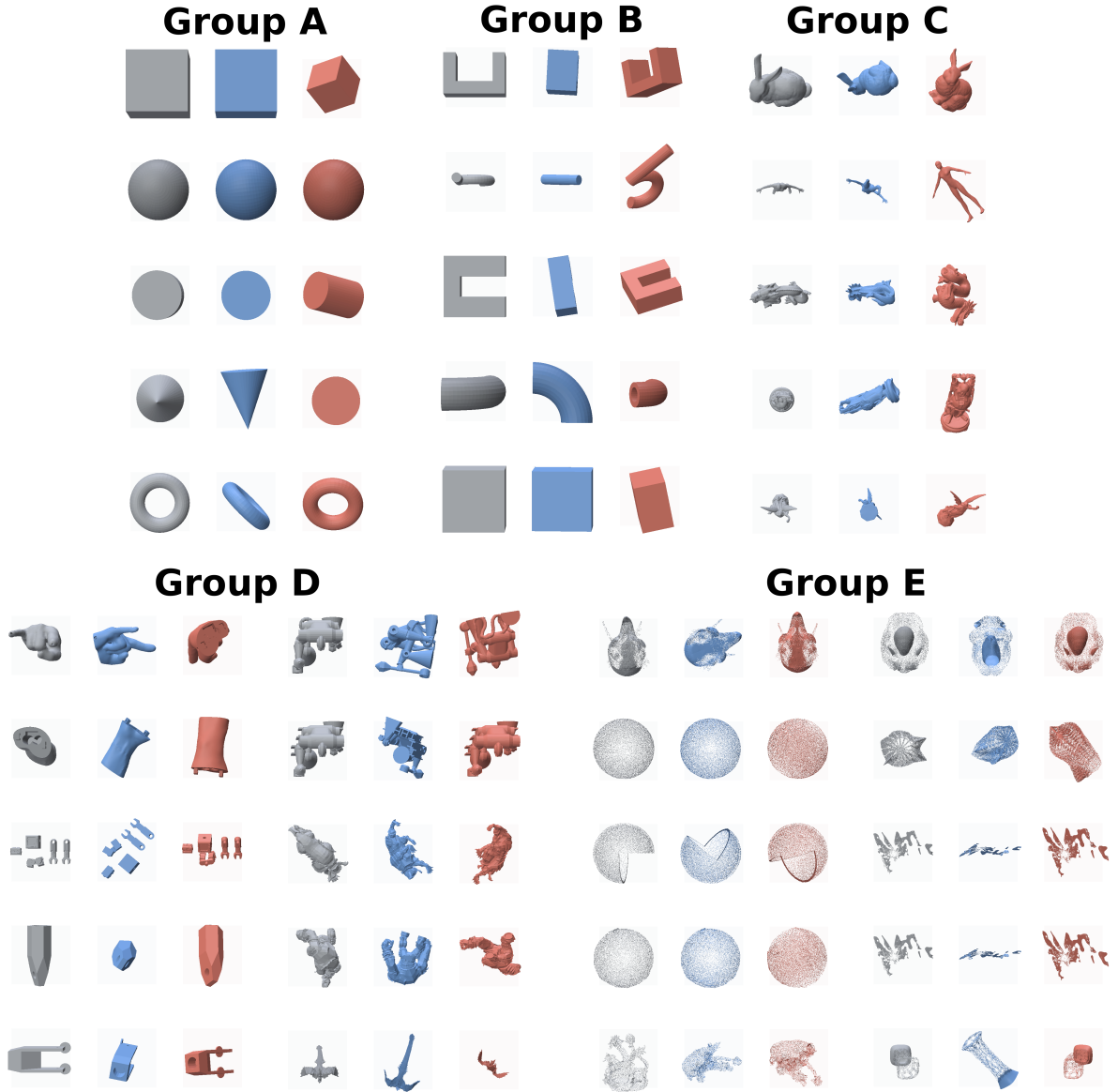


Figure 6: 다섯 그룹(A-E)을 그룹별로 배치한 best/worst 출력 배향 Polyscope 오버레이($\theta_c = 60^\circ$). 각 그룹 상단에 라벨을 두었으며, 윗줄에 그룹 A-C, 아랫줄에 그룹 D/E(각 5행 2열의 세로 배치)를 페이지 중앙 기준으로 묶어 배치하였다. 각 메쉬는 원본(회색), 최적 배향(파랑), 최악 배향(빨강)을 나란히 보이며, 저장된 빌드 방향을 세계 +Z로 회전시켜 실제 출력 자세로 표시하였다. SFTF 후보는 θ_c 에 불변이므로 다섯 임계각의 오버레이는 동일하다.

13.7 최적 배향의 일치성

자명한 사례에서 두 방법은 일치한다. cube, cylinder, cone, u_bracket, c_clamp, hollow_box, pipe_elbow에서 TOMO_INT3 전역 최적 $v_{ss} = 0$ (지지 불필요)이며, SFTF 최상위 방향도 거의 0에 가까운 점수의 축 정렬 면법선 방향을 선택한다. 즉 두 방법 모두 “평면을 빌드플레이트에 안착시키면 지지가 불필요”라는 동일한 결론에 도달한다. 비자명한 유기적/실체 부품(hook, C-D 그룹)에서는 TOMO 전역 최적치 비축 배향에 위치하고 v_{ss} 가 크며, SFTF 역시 높은 hit과 균형 잡힌 R, P, B 로 복잡한 자기지지 구조를 반영한다. 다만 본 5-그룹 실험은 정확도의 정량 ratio가 아니라 계산 특성과 구조 해석의 대비에 초점이 있으며, 방향 정확도의 정량 분석은 §12.1-§12.4의 grid 비교가 담당한다.

요약하면, 본 5-그룹 검증은 다음 SFTF 특성을 정량적으로 확인한다. (i) 전체 A-E에서 Python SFTF는 TOMO_INT3(3° sweep) 대비 1.4-34×, SFTF_C++는 10-5,160×의 계산 시간 단축(더 조밀한 1° grid에서는 격차가 더 커짐), (ii) 메쉬 복잡도에 적응하는 비용(고정비용 부재)과 $\sim 10^6$

면 메쉬까지의 확장성, (iii) 단일 v_{ss} 가 제공하지 못하는 방향별 $R/P/B/\text{hit}$ 구조 분해, (iv) 자명 사례에서 TOMO 전역 최적과의 일치, (v) 네 구현 비교에서 TOMO_CUDA가 항상 INT3보다 빠른 것은 아니며, “평가 대상 축소”라는 알고리즘 축의 이득이 “전수 sweep backend 변경”보다 안정적이고 크다는 점, (vi) C++ 포팅으로 소형 메쉬는 ms 단위, 대형 메쉬도 수 초 단위 후보 평가가 가능함, (vii) 임계각 θ_c 가 커질수록 TOMO 지지체적이 단조 감소하고 전역 최적 배향이 이동하는 반면, 현재 SFTF 후보는 θ_c 에 불변임(§13.6)을 확인하였다.

14 결론

본 연구는 적층제조 출력 방향 최적화를 위해 Support Flow Tensor Field와 빌드플레이트 penalty를 결합한 방향 평가 방법을 제안하였다. 기존 대칭 텐서의 eigenvector 해석은 형상 방향성에는 유용하지만, 방향 의존적인 오버행, 내부 가시성, 빌드플레이트 지지 요구량을 충분히 반영하지 못한다.

현재 알고리즘은 각 후보 방향에 대해 비대칭 Support Flow Tensor를 직접 계산하고, Rayleigh support cost, 면-면 지지량, 빌드플레이트 penalty를 결합한 기본 목적함수 $J = R + P + B$ 를 구성한다. 실제 최종 후보 선택에서는 coarse 구면 샘플링과 소각도 refined sampling으로 만든 후보 풀에 대해 $J, R, P, B, H, S, \sigma_1$ 의 rank-normalized tuning score T 를 적용하여 슬라이서/TOMO 검증에 넘길 유망 배향을 고른다.

정량 분석(§12.1–§12.4)은 본 방법의 강점과 한계를 함께 보여준다. 교차검증된 SFTF는 PCA 및 Support Tensor 고유벡터 방향보다 일관되게 우수하여 출발 가정을 뒷받침한다. 동시에 예측력의 대부분은 빌드플레이트 penalty와 내부 가시성 평가에서 비롯되며, nuclear/ σ_1 등 일부 텐서 파생항의 한계 기여는 작고, 보정된 rank tuning 가중치는 보지 못한 메쉬에 대해 낙관 편향을 보인다. 한편 §5.1–§12.2에서 빌드플레이트 penalty가 흐름 텐서에 가상 접지 노드를 더한 Rayleigh 기여와 정확히 동치임을 보였다. 이로써 예측력을 주도하는 항이 텐서 수축 $\tilde{R} = R + B$ 로 통합되어 “텐서” 정식화가 정당해지며, 음의 신호인 면-면 지지량 P 를 제외한 $\tilde{R}$ 가 기존 J 보다 일관되게 우수함을 확인하였다.

따라서 향후 연구에서는 (i) 가중치를 보정셋이 아닌 교차검증 기반으로 결정하여 과적합을 제어하고, (ii) 기여가 미미한 항을 제거하거나 면-면 self-support coverage, support-volume proxy 등 실제 v_{ss} 와 상관이 높은 feature로 대체하며, (iii) 목적함수 항들의 차원을 정규화하여 가중치의 물리적 의미와 전이성을 확보하고, (iv) projection area, 출력 높이, 접촉 안정성 항을 추가하여 다목적 방향 최적화 모델로 확장하며, (v) §13.6에서 관찰한 대로 TOMO 최적 배향이 임계각 θ_c 에 따라 이동하는 반면 현재 SFTF 후보는 θ_c 에 불변이므로, 오버행 절단을 켜 θ_c 의존 후보 생성으로 확장할 예정이다. 또한 후보 생성이 near-optimal 방향을 포함하지 못하는 경우(예: happy)에 대한 적응적 샘플링도 함께 다룬다.

A TOMO_INT3 v_{ss} 등고선 지형

본 부록은 본문의 비교 기준으로 사용한 저장된 TOMO_INT3 v_{ss} grid (MeshData/*_tomo_int3.vss_grid.1deg_60)에 INT3와 SFTF의 최적/최악 배향을 함께 겹쳐, 두 방법의 예측이 전수 탐색 지형 위에서 어디에 놓이는지를 직접 비교한다. 각 grid는 yaw-pitch 1° 해상도, 임계각 60°에서 계산된 전수 sweep v_{ss} 이며, 361×361 격자 전체를 RdBu_r 발산형 colormap의 채워진 등고선으로 표시하였다 (파란색일수록 v_{ss} 가 작은 좋은 배향, 빨간색일수록 지지구조량이 큰 배향, colormap은 본문 HTML 그림과 동일하다). 네 종류의 마커를 각각 3개씩 겹쳐 표시한다: 별표(*, INT3 optimal; 격자에서 v_{ss} 가 가장 작은 서로 다른 세 basin), X(INT3 worst; v_{ss} 가 가장 큰 세 basin), 원(SFTF optimal; SFTF_C++ 상위 세 방향 sftf_cpp_optimal), 다이아몬드(SFTF worst; 하위 세 방향 sftf_cpp_worst)이며, SFTF 방향은 각자 가장 가까운 격자 셀에 배치하였다. 범례는 그림을 가리지 않도록 각 그림 아래에 두었다. 따라서 SFTF optimal(원)이 INT3 optimal(별표)에 가깝고 SFTF worst(다이아몬드)가 INT3 worst(X)에 가까울수록, 수천 배 저렴한 SFTF가 전수 탐색의 좋은/나쁜 배향을 잘 재현함을 뜻한다. 이 정적 그림들은 저장된 npz 격자와 검증 기록(Group_C*.json)에서 scripts/render_appendix_contours.py로 렌더링하였다.

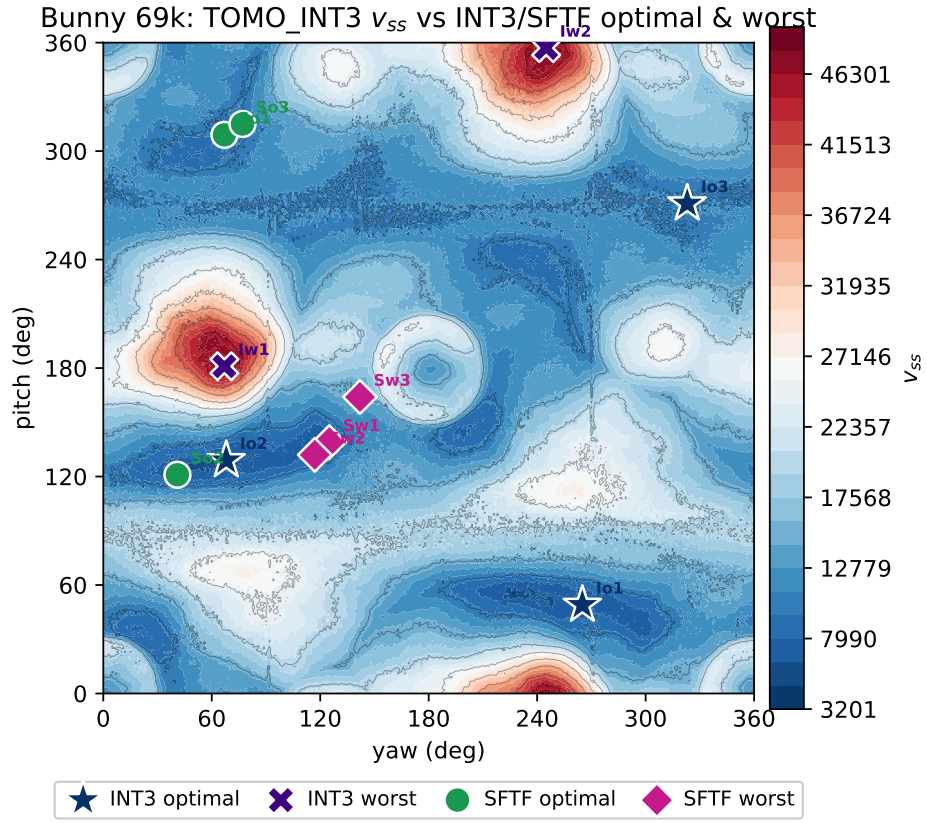


Figure 7: Bunny 69k의 TOMO_INT3 v_{ss} 지형 위 INT3/SFTF 최적 · 최악 배향 비교(파랑이 좋음). 마커는 INT3/SFTF 각 optimal · worst 3개씩이며 종류는 그림 아래 범례를 따른다. 전역 최적은 $v_{ss} = 3201$ (yaw= 265°, pitch= 49°; 본문 Table 1)이며, 단일 점이 아니라 넓은 저비용 basin 안에 위치한다.

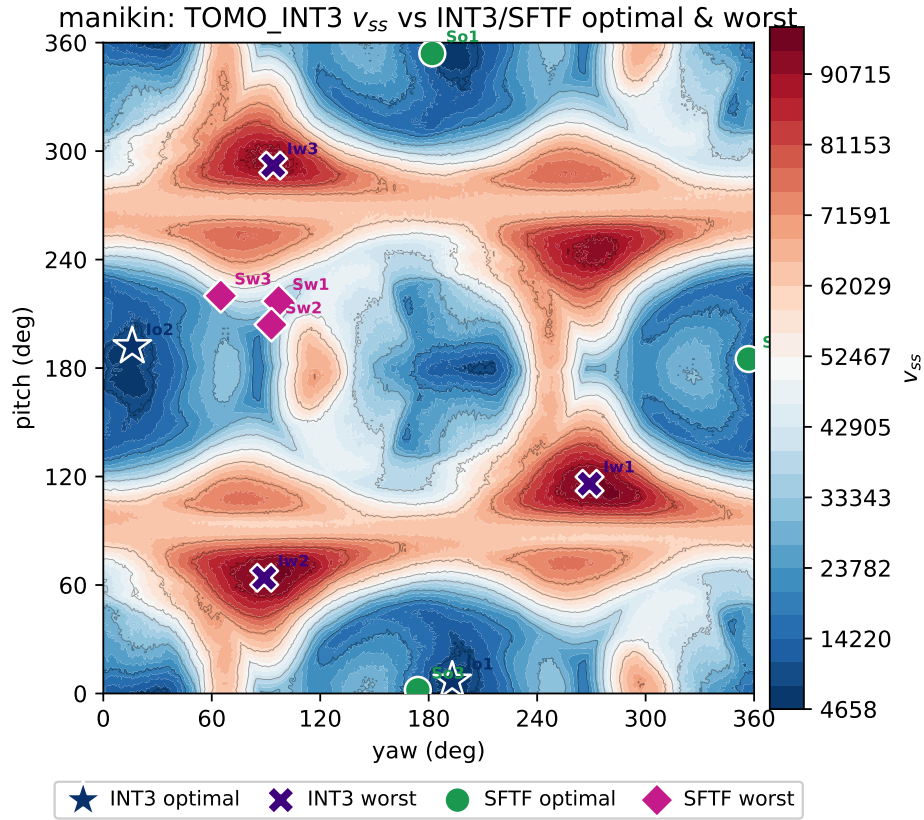


Figure 8: manikin의 TOMO_INT3 v_{ss} 지형 위 INT3/SFTF 최적 · 최악 배향 비교(파랑이 좋음). 마커는 INT3/SFTF 각 optimal · worst 3개씩이며 종류는 그림 아래 범례를 따른다. 전역 최적은 $v_{ss} = 4658$ (yaw= 193°, pitch= 7°)이고, 세로로 긴 형상답게 저비용 영역이 yaw 전 구간의 좁은 pitch 띠에 분포한다.

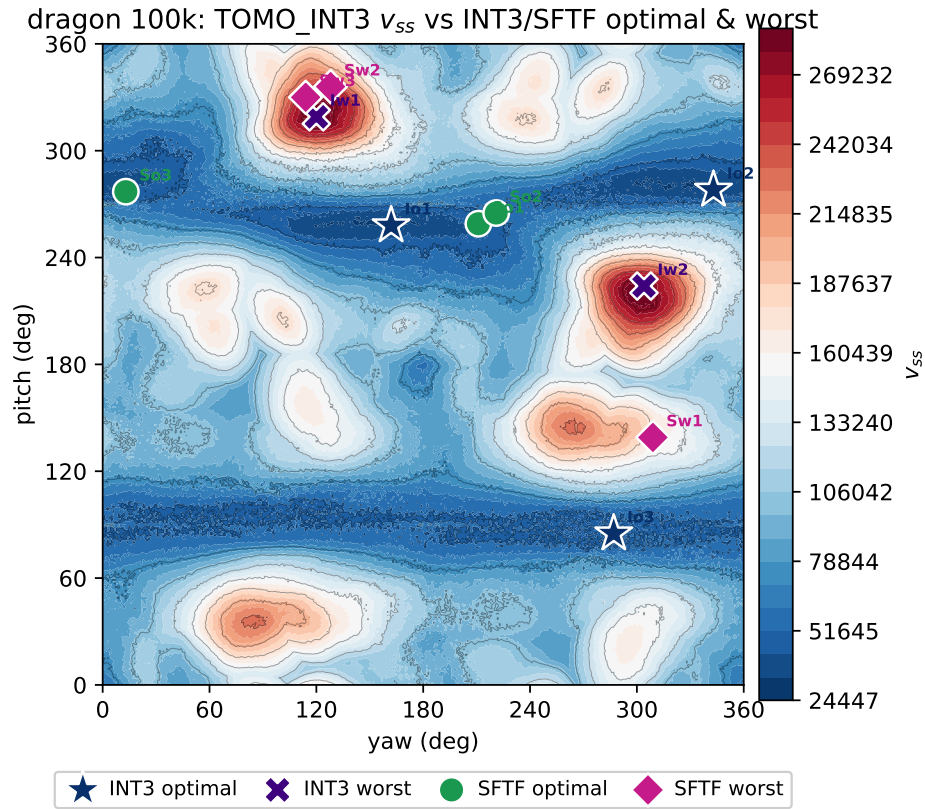


Figure 9: dragon 100k의 TOMO_INT3 v_{ss} 지형 위 INT3/SFTF 최적 · 최악 배향 비교(파란이 좋음). 마커는 INT3/SFTF 각 optimal · worst 3개씩이며 종류는 그림 아래 범례를 따른다. 전역 최적은 $v_{ss} = 24447$ (yaw= 162°, pitch= 258°)이며, 표면 디테일이 많아 절대 v_{ss} 규모가 크지만 전역 최적은 비교적 뚜렷한 basin을 형성한다.

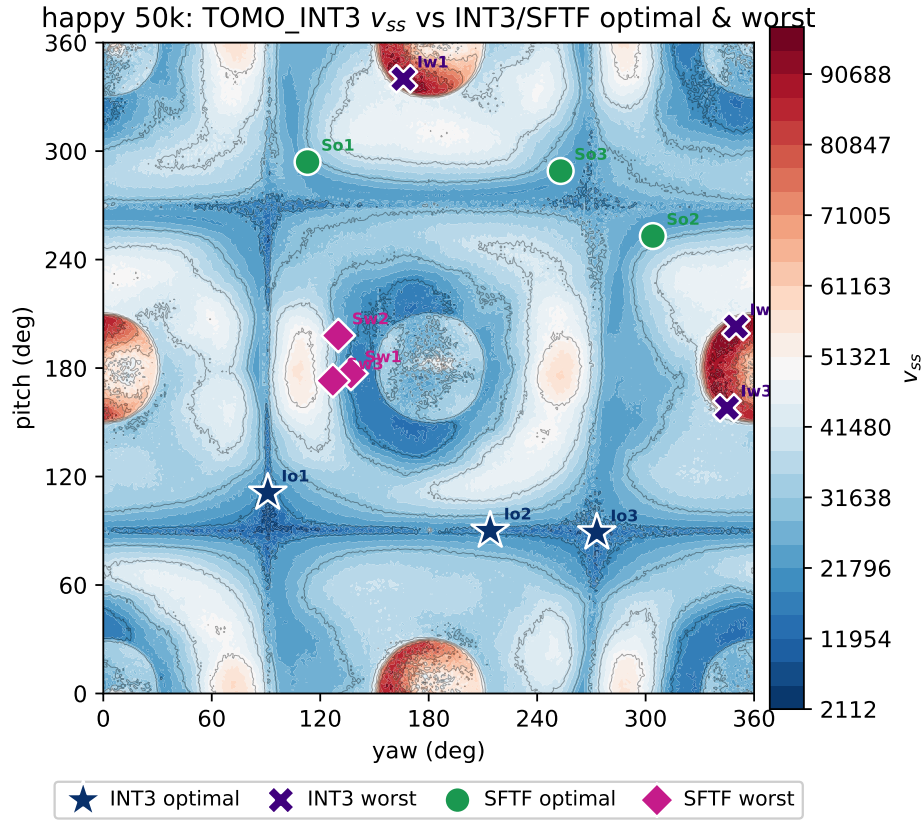


Figure 10: happy 50k의 TOMO_INT3 v_{ss} 지형 위 INT3/SFTF 최적 · 최악 배향 비교(파랑이 좋음). 마커는 INT3/SFTF 각 optimal · worst 3개씩이며 종류는 그림 아래 범례를 따른다. 전역 최적은 $v_{ss} = 2112$ (yaw= 91°, pitch= 111°)이다. §12.5에서 논의한 난해 사례로, 저비용 영역이 좁고 분산되어 있어 기본 512 방향 후보 풀이 near-optimal 배향을 포함하지 못하며, SFTF optimal(원)도 INT3 optimal(별표)에서 떨어진 곳에 형성된다(Table 8).

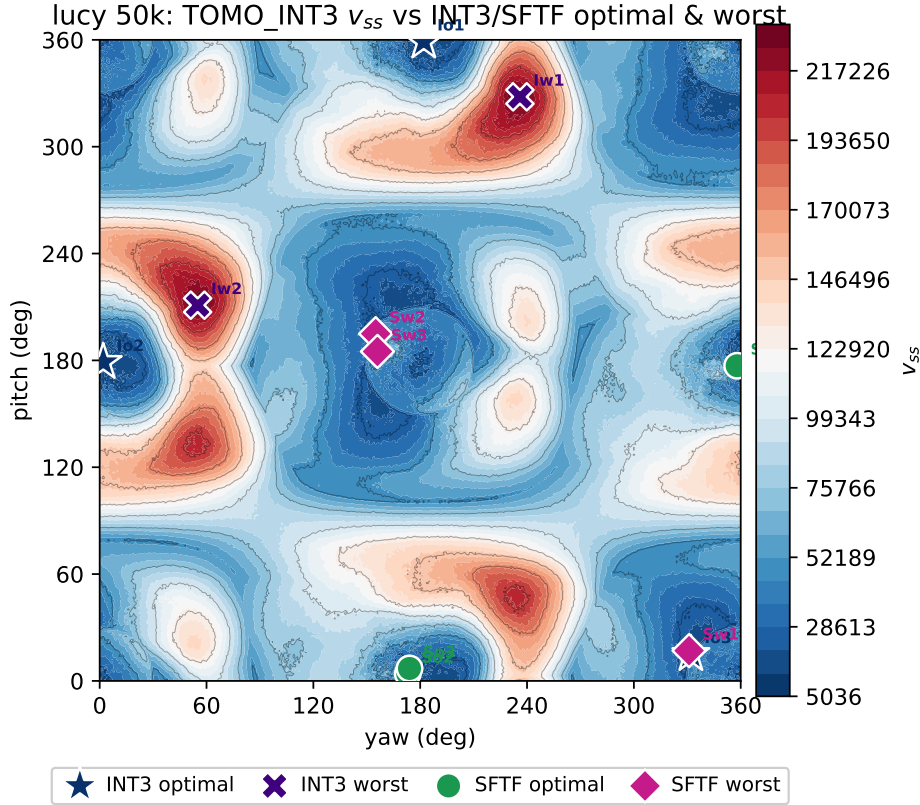


Figure 11: lucy 50k의 TOMO_INT3 v_{ss} 지형 위 INT3/SFTF 최적 · 최악 배향 비교(파랑이 좋음). 마커는 INT3/SFTF 각 optimal · worst 3개씩이며 종류는 그림 아래 범례를 따른다. 전역 최적은 $v_{ss} = 5036$ (yaw= 182°, pitch= 359°)이며, pitch 경계(359°) 부근에 위치하여 yaw-pitch 주기성을 고려한 basin 탐색이 필요하다.

이 다섯 지형은 본문의 관찰을 시각적으로 뒷받침한다. Bunny와 dragon처럼 전역 최적이 넓은 basin 안에 있는 경우 coarse 구면 샘플링만으로도 near-optimal 배향이 후보 풀에 쉽게 포함되는 반면, happy처럼 저비용 영역이 좁고 여러 곳에 흩어진 경우 샘플링 해상도가 병목이 된다. 또한 lucy처럼 최적의 yaw-pitch 경계에 위치하는 경우는 본문에서 강조한 주기성을 고려한 basin 표현의 필요성을 보여준다.

References

- [1] Fan R. K. Chung. *Spectral Graph Theory*. Number 92 in CBMS Regional Conference Series in Mathematics. American Mathematical Society, 1997.
- [2] William G. Cochran. *Sampling Techniques*. Wiley Series in Probability and Statistics. Wiley, New York, 3 edition, 1977.
- [3] Chengkai Dai, Charlie C. L. Wang, Chenming Wu, Sylvain Lefebvre, Guoxin Fang, and Yong-Jin Liu. Support-free volume printing by multi-axis motion. *ACM Transactions on Graphics*, 37(4):1–14, 2018.
- [4] Paramita Das, Kunal Mhapsekar, Souran Chowdhury, Rutuja Samant, and Sam Anand. Selection of build orientation for optimal support structures and minimum part errors in additive manufacturing. *Computer-Aided Design and Applications*, 14(sup1):1–13, 2017.
- [5] Ben Ezair, Fady Massarwi, and Gershon Elber. Orientation analysis of 3D objects toward minimal support volume in 3D-printing. *Computers & Graphics*, 51:117–124, 2015.

- [6] Harald Garcke, Kei Fong Lam, Robert Nürnberg, and Andrea Signori. Overhang penalization in additive manufacturing via phase field structural topology optimization with anisotropic energies. *Applied Mathematics & Optimization*, 87(3):44, 2023.
- [7] Andrew T. Gaynor, Nicholas A. Meisel, Christopher B. Williams, and James K. Guest. Topology optimization for additive manufacturing: Considering maximum overhang constraint. In *15th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, page 2036, 2014.
- [8] Ian Gibson, David Rosen, and Brent Stucker. *Additive Manufacturing Technologies*. Springer, New York, 2 edition, 2015.
- [9] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013.
- [10] Álvaro González. Measurement of areas on a sphere using Fibonacci and latitude–longitude lattices. *Mathematical Geosciences*, 42(1):49–64, 2010.
- [11] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Waltham, MA, 3 edition, 2011.
- [12] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [13] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, 2 edition, 2013.
- [14] Jingchao Jiang, Jonathan Stringer, Xun Xu, and Ray Y. Zhong. Investigation of printable threshold overhang angle in extrusion-based additive manufacturing for reducing support waste. *International Journal of Computer Integrated Manufacturing*, 31(10):961–969, 2018.
- [15] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics. Springer, New York, 2 edition, 2002.
- [16] Jae Yeol Jung, Seokjae Chee, and In Hwan Sul. Support structure tomography using per-pixel signed shadow casting in human manikin 3D printing. *Fashion and Textiles*, 9(1):1–18, 2022.
- [17] Ken-Ichi Kanatani. Distribution of directional data and fabric tensors. *International Journal of Engineering Science*, 22(2):149–164, 1984.
- [18] Jae Ryoung Kim and In Hwan Sul. Fast prediction of 3D printing optimal orientation using general-purpose graphic processor unit calculation. *3D Printing and Additive Manufacturing*, 13(1):50–62, 2026.
- [19] Matthijs Langelaar. Combined optimization of part topology, support structure layout and build orientation for additive manufacturing. *Structural and Multidisciplinary Optimization*, 57(5):1985–2004, 2018.
- [20] K. V. Mardia and P. E. Jupp. *Directional Statistics*. Wiley Series in Probability and Statistics. Wiley, Chichester, 2000.
- [21] Amir M. Mirzendehtdel and Krishnan Suresh. Support structure constrained topology optimization for additive manufacturing. *Computer-Aided Design*, 81:1–13, 2016.
- [22] Tomas Möller and Ben Trumbore. Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.

- [23] Scott D. Roth. Ray casting for modeling solids. *Computer Graphics and Image Processing*, 18(2):109–144, 1982.
- [24] E. B. Saff and A. B. J. Kuijlaars. Distributing many points on a sphere. *The Mathematical Intelligencer*, 19(1):5–11, 1997.
- [25] Seungheon Shin, Yujin Park, and Gil Ho Yoon. Topology optimization for multi-axis additive manufacturing considering overhang and anisotropy. *International Journal of Mechanical Sciences*, 301:110443, 2025.
- [26] Robert Smith and Michael C. Hollenbeck. Build orientation for additive manufacturing of complex structures. U.S. Patent No. 12,343,928, 2025. Assignee: Optisys, Inc.
- [27] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.
- [28] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.
- [29] Benjamin Vaissier, Jean-Philippe Pernot, Laurent Chougrani, and Philippe Véron. Genetic-algorithm based framework for lattice support structure optimization in additive manufacturing. *Computer-Aided Design*, 110:11–23, 2019.
- [30] Charlie C. L. Wang, Yong-Shun Leung, and Yong Chen. Solid modeling of polyhedral objects by layered depth-normal images on the GPU. *Computer-Aided Design*, 42(6):535–544, 2010.
- [31] Qingnan Zhou and Alec Jacobson. Thingi10k: A dataset of 10,000 3D-printing models. *arXiv preprint arXiv:1605.04797*, 2016.